VOL. 27

Kubernetes

Clusters, workloads and the declarative object model that runs modern applications

Source	kubernetes.io/docs/tutorials
Generated	2026-10-10
Level	Intermediate
Estimated study time	30 hours
Language	English
Volume	Vol. 27 - Kubernetes

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 8

Chapter map 8

Chapter 1 - Kubernetes architecture: control plane, nodes and objects 9

1.1 Summary 9

1.2 Concepts 10

The two planes 10

What each component is for 11

Everything is an object 11

Labels and selectors 12

The reconciliation loop 12

Namespaces 12

1.3 Examples 13

1.4 Common mistakes and best practices 15

1.5 Practice exercises 15

Solutions 16

1.6 Mini test 16

Answer key and explanations 16

Chapter 2 - Pods: the smallest unit you can deploy 17

2.1 Summary 17

2.2 Concepts 17

What a pod really is 17

Lifecycle and restart policy 18

The four multi-container patterns 18

2.3 Examples 19

2.4 Common mistakes and best practices 21

2.5 Practice exercises 22

Solutions 22

2.6 Mini test 22

Answer key and explanations 23

Chapter 3 - ReplicaSets and Deployments 23

3.1 Summary 23

3.2 Concepts 24

The ownership chain . . . 24
What a ReplicaSet does . . . 24
What a Deployment adds . . . 24
Selectors and templates . . . 25
3.3 Examples . . . 25
3.4 Common mistakes and best practices . . . 28
3.5 Practice exercises . . . 28
Solutions . . . 28
3.6 Mini test . . . 29
Answer key and explanations . . . 29
Chapter 4 - Services and cluster DNS . . . 29
4.1 Summary . . . 29
4.2 Concepts . . . 30
Why Services exist . . . 30
The four Service types . . . 30
Ports and targetPort . . . 31
Endpoints . . . 31
Cluster DNS . . . 31
4.3 Examples . . . 31
4.4 Common mistakes and best practices . . . 34
4.5 Practice exercises . . . 34
Solutions . . . 35
4.6 Mini test . . . 35
Answer key and explanations . . . 35
Chapter 5 - ConfigMaps and Secrets . . . 36
5.1 Summary . . . 36
5.2 Concepts . . . 36
Where configuration belongs . . . 36
Three ways to consume values . . . 36
Volumes are files, not values . . . 37
5.3 Examples . . . 37
5.4 Common mistakes and best practices . . . 40
5.5 Practice exercises . . . 40
Solutions . . . 41
5.6 Mini test . . . 41

Answer key and explanations . . . 41
Chapter 6 - Storage: volumes, PVs, PVCs and StatefulSets . . . 42
6.1 Summary . . . 42
6.2 Concepts . . . 42
The storage chain . . . 42
Volume kinds you will meet . . . 42
Access modes and policies . . . 43
PV and PVC matching . . . 43
StatefulSets . . . 43
6.3 Examples . . . 44
6.4 Common mistakes and best practices . . . 47
6.5 Practice exercises . . . 47
Solutions . . . 47
6.6 Mini test . . . 48
Answer key and explanations . . . 48
Chapter 7 - Probes, requests and limits . . . 48
7.1 Summary . . . 48
7.2 Concepts . . . 49
The three probes . . . 49
Requests and limits . . . 50
QoS classes . . . 50
Quota and defaults . . . 50
7.3 Examples . . . 51
7.4 Common mistakes and best practices . . . 54
7.5 Practice exercises . . . 54
Solutions . . . 55
7.6 Mini test . . . 55
Answer key and explanations . . . 55
Chapter 8 - Namespaces, RBAC and pod security . . . 56
8.1 Summary . . . 56
8.2 Concepts . . . 56
Namespaces as the unit of tenancy . . . 56
RBAC: four verbs, three ideas . . . 56
Pod Security Standards . . . 57
The security context . . . 58

8.3 Examples . . . 58
8.4 Common mistakes and best practices . . . 60
8.5 Practice exercises . . . 61
Solutions . . . 61
8.6 Mini test . . . 61
Answer key and explanations . . . 62
Chapter 9 - Ingress and external access . . . 62
9.1 Summary . . . 62
9.2 Concepts . . . 63
Where Ingress sits . . . 63
Ingress versus Services . . . 63
Anatomy of an Ingress . . . 63
TLS at the edge . . . 64
9.3 Examples . . . 64
9.4 Common mistakes and best practices . . . 66
9.5 Practice exercises . . . 67
Solutions . . . 67
9.6 Mini test . . . 67
Answer key and explanations . . . 68
Chapter 10 - Helm and packaging . . . 68
10.1 Summary . . . 68
10.2 Concepts . . . 69
What a chart contains . . . 69
Values and the template context . . . 69
The release model . . . 70
10.3 Examples . . . 70
10.4 Common mistakes and best practices . . . 73
10.5 Practice exercises . . . 74
Solutions . . . 74
10.6 Mini test . . . 74
Answer key and explanations . . . 75
Chapter 11 - Observability basics . . . 75
11.1 Summary . . . 75
11.2 Concepts . . . 75

The default troubleshooting path . . . 75
The sidecar logging pattern . . . 76
What is built in and what is not . . . 76
11.3 Examples . . . 77
11.4 Common mistakes and best practices . . . 79
11.5 Practice exercises . . . 79
Solutions . . . 79
11.6 Mini test . . . 80
Answer key and explanations . . . 80
Chapter 12 - Rolling updates, rollbacks and release strategies . . . 80
12.1 Summary . . . 80
12.2 Concepts . . . 81
The rollout controller . . . 81
The strategy numbers . . . 81
Release strategies compared . . . 81
12.3 Examples . . . 82
12.4 Common mistakes and best practices . . . 84
12.5 Practice exercises . . . 85
Solutions . . . 85
12.6 Mini test . . . 85
Answer key and explanations . . . 86
Cheat sheet . . . 86
13.1 Cluster and objects . . . 86
13.2 Object anatomy . . . 86
13.3 Pods and workloads . . . 87
13.4 Services . . . 87
13.5 Configuration and storage . . . 87
13.6 Probes and resources . . . 88
13.7 Security quick reference . . . 88
13.8 Helm . . . 88
13.9 Deploy and roll back . . . 89
13.10 Troubleshooting order . . . 89
Final test . . . 89
14.1 Final test answers . . . 91

Sources **92**

How to use this volume

Prerequisites. A Linux or macOS command line, the basics of containers (what an image is, what a container runtime does), and enough YAML to read an indented list. Docker knowledge helps but is not required: Kubernetes starts containers for you. No previous Kubernetes experience is assumed.

Estimated study time. About 30 hours: two to three hours per chapter, plus the time you spend experimenting. Chapters 1 to 7 are the core you will use every day. Chapters 8 to 12 cover security, packaging and operations.

How the chapters work. Every chapter opens with a summary, explains the concepts with diagrams, shows numbered examples with the output they produce, lists common mistakes next to the best practice, and ends with practice exercises, their solutions, and a mini test with an answer key.

How to run the code. Manifests in this volume are ordinary YAML files: save one as `pod.yaml` and apply it with `kubectl apply -f pod.yaml`. Blocks tagged `bash` are terminal commands, blocks tagged `yaml` are manifests you keep in version control, and blocks tagged `text` show what appears on screen.

shell

```
minikube start --driver=docker
kind create cluster --name study
kubectl get nodes
```

Use either of the first two commands, whichever cluster tool you installed, then let `kubectl get nodes` confirm the result.

output

```
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   42s   v1.31.0
```

About the command output. The volume was written on a machine without a running cluster, so outputs of commands that talk to a cluster are marked not executed. They show the shape of real output on a healthy cluster; column widths, timestamps, IP addresses and versions will differ on your machine. Every manifest, on the other hand, is complete and was validated as syntactically correct YAML with the required fields for its kind.

About the source. The syllabus follows the official Kubernetes tutorials at kubernetes.io/docs/tutorials and the concept pages they link to. All text, diagrams, comments and exercises here are original.

Chapter map

#	Chapter	You will learn
1	Kubernetes architecture: control plane, nodes and objects	how a cluster is assembled, what each component does, and how desired state becomes running containers

#	Chapter	You will learn
2	Pods: the smallest unit you can deploy	pod anatomy, shared namespaces, multi-container patterns, logs and exec
3	ReplicaSets and Deployments	controllers, label selectors, ownership, scaling and self-healing
4	Services and cluster DNS	stable virtual IPs, endpoints, ClusterIP, NodePort, headless and ExternalName
5	ConfigMaps and Secrets	injecting configuration as environment variables and mounted files
6	Storage: volumes, PVs, PVCs and StatefulSets	emptyDir, persistent volumes, claims, storage classes, stable identities
7	Probes, requests and limits	liveness, readiness and startup probes, requests, limits and QoS
8	Namespaces, RBAC and pod security	namespace scoping, roles and bindings, service accounts, Pod Security Standards
9	Ingress and external access	ingress controllers, path and host routing, TLS, LoadBalancer services
10	Helm and packaging	chart anatomy, values files, releases, upgrade and rollback
11	Observability basics	logs, events, metrics, and the sidecar logging pattern
12	Rolling updates, rollbacks and release strategies	rollout control, canary and blue/green, safe deploys
-	Cheat sheet	compact reference for the whole volume
-	Final test	25 questions with answers and explanations

Chapter 1 - Kubernetes architecture: control plane, nodes and objects

1.1 Summary

- Kubernetes runs containers across a fleet of machines and keeps them in the state you describe.
- A cluster is one control plane plus any number of worker nodes, each of which is a host running a container runtime.
- The control plane stores the desired state, decides where new work goes, and endlessly reconciles reality with that desired state.
- Everything you manage is an API object with metadata, a spec you write, and a status the system fills in.

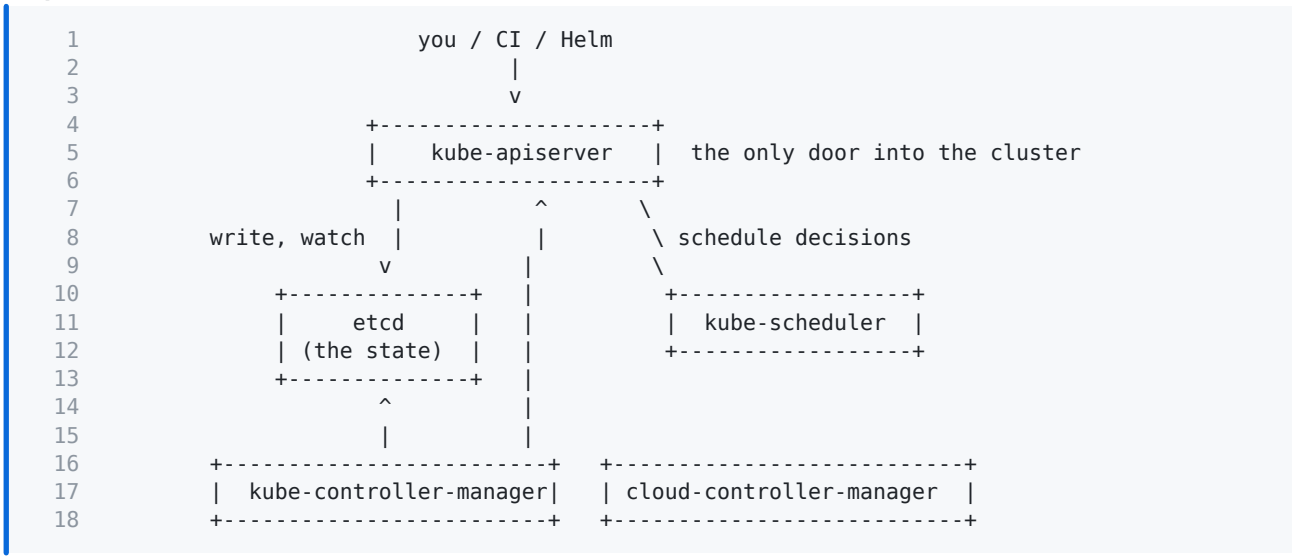
- You talk to the cluster through `kubectl`, which sends requests to the API server; the API server is the only component that reads and writes the datastore.
- Labels and selectors are the glue that connects a controller to the objects it manages.
- Every controller runs the same loop: observe, compare with the desired state, act, repeat.
- Namespaces split one cluster into separate scopes for teams, environments or projects.
- A local single-node cluster is enough to learn, and `minikube` or `kind` produce a real, standards-compliant one.
- Because everything is an API resource, the same declarative workflow covers applications, storage, security and networking.

1.2 Concepts

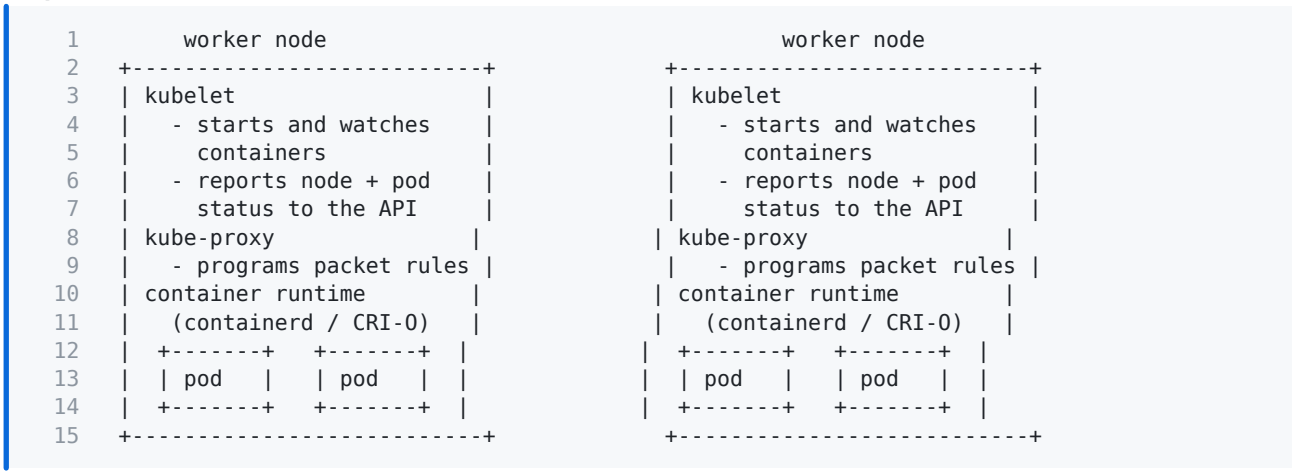
The two planes

A cluster has exactly one control plane and one or more nodes. The control plane is the brain: it holds the state and makes decisions. The nodes are the muscle: they start and stop containers and report back.

output



output



What each component is for

Component	Lives on	Job
kube-apiserver	control plane	serves the REST API, validates objects, writes them to the datastore
etcd	control plane	the consistent key-value store that holds every object
kube-scheduler	control plane	picks a node for each new pod, using requests, limits and rules
kube-controller-manager	control plane	runs the built-in controllers (deployments, nodes, endpoints, ...)
cloud-controller-manager	control plane	talks to the cloud API for load balancers, disks, routes
kubelet	every node	makes sure the pods assigned to its node are running and healthy
kube-proxy	every node	implements the virtual IPs of Services with packet-filter rules
container runtime	every node	pulls images and starts containers (containerd, CRI-O)
CoreDNS	cluster add-on	answers the DNS names of Services and pods

Only the API server touches etcd. Controllers and kubelets watch the API and act on what they see; that is why every tool, including `kubectl`, speaks HTTP to one endpoint.

Everything is an object

Kubernetes exposes its whole world as API objects. Each object has the same three outer blocks, plus a `status` the system owns:

YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: demo
  labels:
    app: demo
spec:
  containers:
    - name: web
      image: nginx:1.27
status:
  phase: Running
```

Field	Written by	Meaning
`apiVersion`	you	group and version of the API that owns the object
`kind`	you	the type: Pod, Deployment, Service, ConfigMap, ...
`metadata`	you	name, namespace, labels, annotations, owner references

Field	Written by	Meaning
`spec`	you	the desired state
`status`	the system	what is true right now

Names are unique inside a namespace. Most objects also carry labels, which are arbitrary key-value pairs that controllers and Services use to find them.

Labels and selectors

Labels solve a single problem: loosely binding things together. A Service binds to pods by selector, and a Deployment owns the pods whose labels match its selector template.

YAML

```yaml
metadata:
  labels:
    app: web
    tier: frontend
    release: stable
```

YAML

```yaml
selector:
  matchLabels:
    app: web
```

Annotations look like labels but are for non-identifying metadata: timestamps, checksums, tool configuration. Selectors never match on annotations.

The reconciliation loop

The cluster is described, never commanded. You say "three web replicas"; the controller counts what exists and creates or deletes pods until the numbers match. This loop is the heart of Kubernetes.

output

```
      desired state            observed state
      +-----------+            +-----------+
      | replicas:3|            | replicas:2|
      +-----+-----+            +-----+-----+
            |                        |
            +----------+-------------+
                       v
             controller sees the gap
                       |
                       v
             create one more pod
                       |
                       v
             observe again, forever
```

Namespaces

A namespace is a scope inside one cluster. Names must be unique within their namespace, quotas and role bindings apply per namespace, and the `default` namespace is where your

objects land when you do not specify one. The system creates `kube-system`, `kube-public`, `kube-node-lease` and `default` for you.

1.3 Examples

Example 1: create a local single-node cluster (not executed).

shell

```
minikube start --driver=docker --kubernetes-version=v1.31.0
```

output

```
* minikube v1.34.0 on Microsoft Windows 11
* Using the docker driver based on the user configuration
* Starting control plane node minikube in cluster minikube
* Pulling base image v1.31.0 ...
* Creating docker container (CPUs=2, Memory=3800MB) ...
* Preparing Kubernetes v1.31.0 on Docker 27.2.0 ...
    - Generating certificates and keys
    - Booting up control plane
    - Configuring RBAC rules
* Verifying Kubernetes components ...
* Enabled addons: storage-provisioner, default-storageclass
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace
```

`kind create cluster --name study` produces an equivalent cluster; both run a real control plane in containers on your machine.

Example 2: confirm the cluster answers (not executed).

shell

```
kubectl get nodes
kubectl get namespaces
```

output

```
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   62s   v1.31.0

NAME              STATUS   AGE
default           Active   68s
kube-node-lease   Active   68s
kube-public       Active   68s
kube-system       Active   68s
```

Example 3: create a namespace from a manifest.

YAML

```
apiVersion: v1
kind: Namespace
metadata:
  name: study
  labels:
    purpose: learning
```

shell

```
kubectl apply -f namespace.yaml
kubectl get namespaces
```

output

```
namespace/study created

NAME              STATUS   AGE
default           Active   6m
kube-node-lease   Active   6m
kube-public       Active   6m
kube-system       Active   6m
study             Active   4s
```

Example 4: ask the API server to document itself.

shell

```
kubectl explain pod.spec.containers
kubectl api-resources | head -5
```

output

```
KIND:     Pod
VERSION:  v1
FIELD:    containers <[]Object>

DESCRIPTION:
     List of containers belonging to the pod. ...

NAME                              SHORTNAMES   APIVERSION                             NAMESPACED   KIND
bindings                                       v1                                     true         Bind
componentstatuses                 cs           v1                                     false        Comp
configmaps                        cm           v1                                     true         Conf
endpoints                         ep           v1                                     true         Endp
```

`explain` prints the server-side documentation for any field, which is the fastest way to check a field name without leaving the terminal.

Example 5: look at a real object end to end (not executed).

shell

```
kubectl create deployment web --image=nginx:1.27 --replicas=2 -n study
kubectl get deployment web -n study -o yaml | head -20
```

output

```
deployment.apps/web created

apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    deployment.kubernetes.io/revision: "1"
  labels:
    app: web
  name: web
  namespace: study
spec:
  progressDeadlineSeconds: 600
  replicas: 2
```

The output shows the anatomy: `metadata` with labels, a `spec` with the replica count, and, further down, a `status` block the Deployment controller updates.

1.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Editing live objects with `kubectl edit`	nothing records who changed what	change the manifest, apply it, review the diff
Keeping the only copy of a manifest on the server	the cluster cannot be rebuilt	keep YAML in Git
Confusing pods with nodes	you debug the wrong layer	nodes are machines, pods are workloads
Expecting a deleted pod to come back with the same name	pod names are ephemeral	use a Deployment or another controller
One giant manifest per application	reviews are impossible, reuse is nil	one file per object type
Forgetting labels the Service selects on	the Service has no endpoints	label the pod template, not only the Service
Treating `status` as input	your change is overwritten	write `spec`, read `status`

- When something surprises you, `kubectl describe` on the object tells you what the controller last saw.
- Keep `kubectl config current-context` in mind: a command always goes to the cluster and namespace your context points at.

1.5 Practice exercises

1. Which component holds the cluster state, and which is the only component allowed to modify it?
2. List the four field groups that make up every Kubernetes object and say who writes each one.

3. Write a manifest that creates a namespace named `lab` with the label `purpose: experiments`.
4. A Deployment asks for four replicas and three pods exist. Describe what the controller does next.
5. Why is it wrong to store an application's configuration only in the image?

Solutions

1. etcd holds the state, and only kube-apiserver reads and writes it. All other components talk to the API server.
2. `apiVersion` and `kind` (identity, written by you), `metadata` (name, namespace, labels), `spec` (the desired state, written by you), and `status` (the observed state, written by the system).
3. `yaml apiVersion: v1 kind: Namespace metadata: name: lab labels: purpose: experiments`
4. It notices a gap of one and creates a pod from the stored template, then keeps watching in case the number drifts again.
5. An image change is needed for every configuration change, so the two life cycles are coupled; external configuration (ConfigMaps, Secrets, values) lets you ship one immutable image to every environment.

1.6 Mini test

1. Which component is the only one that talks to etcd? A) kube-scheduler B) kubelet C) kube-apiserver D) kube-controller-manager
2. Which block of an object does the control plane fill in? A) `spec` B) `status` C) `metadata.labels` D) `apiVersion`
3. What does a controller do with the observed state? A) It compares it with the desired state and acts on the difference B) It writes it to the container image C) It restarts the node D) It deletes the namespace
4. True or false: pod names are stable for the life of the application.
5. Which two fields does a label selector need to match against? A) annotations on the object B) labels on the object C) the object name D) the namespace name
6. Which tool speaks to the cluster? A) the container runtime, directly B) `kubectl`, over the REST API C) the scheduler, over gRPC D) `docker`, over a unix socket
7. Where does an object go when no namespace is given? A) `kube-system` B) `default` C) it is rejected D) `kube-public`

Answer key and explanations

1. C - the API server is the single door to the datastore, which keeps validation and auditing in one place.
2. B - `status` is a report of what the system observed; you never write it.
3. A - the reconcile loop is the engine that turns desired state into running containers.
4. False - controllers create new pods with new names whenever the old ones disappear.
5. B - selectors match labels; annotations are for non-identifying data.

6. B - `kubectl` is an HTTP client for the same API every other tool uses.
7. B - objects land in `default` unless you pass `-n` or set a namespace in the manifest.

Chapter 2 - Pods: the smallest unit you can deploy

2.1 Summary

- A pod is the smallest object Kubernetes can run, and it wraps one or more containers.
- All containers in a pod share the network namespace, so they reach each other on `localhost`.
- Containers in a pod also share mounted volumes, which is how sidecars read what the main container writes.
- A pod has a lifecycle: `Pending`, `Running`, `Succeeded`, `Failed` and `Unknown`.
- `restartPolicy` decides whether a failed container is restarted inside the same pod.
- Multi-container pods follow patterns: sidecar, init container, ambassador and adapter.
- Init containers run to completion before the application containers start.
- A bare pod is mortal: if it is deleted, nothing recreates it, which is why workloads use controllers.
- `kubectl logs`, `kubectl exec` and `kubectl describe` are the three tools you need to see inside a pod.
- Resource requests and limits, security context and probes are all configured per container inside the pod spec.

2.2 Concepts

What a pod really is

A pod is a small ring of containers that behave like one unit: same node, same IP, same port space, shared volumes. The runtime starts one pause container per pod to hold those shared namespaces; your containers then join them.

output

```
                  +-----------------------------------------+
                  |                  pod                    |
                  |                                         |
    client -----> |  10.244.0.15:80                         |
                  |                                         |
                  |  +-----------+        +---------------+ |
                  |  |  web      |        |  log-shipper  | |
                  |  |  nginx    |------->|  sidecar      | |
                  |  +-----------+        +---------------+ |
                  |        |          reads /var/log via a   |
                  |        |          shared volume          |
                  |  +-----v-----------------------------+   |
                  |  |  volume: shared-logs (emptyDir)   |   |
                  |  +-----------------------------------+   |
                  +-----------------------------------------+
```

Two containers in the same pod talk over `localhost`; they do not need a Service, and they always live on the same node.

Lifecycle and restart policy

Phase	Meaning
`Pending`	accepted, but not all containers are running yet (image pull, scheduling)
`Running`	the pod is bound to a node and at least one container is running
`Succeeded`	every container finished with exit code 0
`Failed`	at least one container exited non-zero
`Unknown`	the node stopped reporting; the control plane cannot say

`restartPolicy` controls what happens when a container exits:

Policy	Behaviour	Typical use
`Always`	restart on any exit	long-running services (the default)
`OnFailure`	restart only on non-zero exit	batch jobs
`Never`	never restart	one-shot tasks

The four multi-container patterns

Pattern	Shape	When to use
Sidecar	helper runs next to the app	shipping logs, proxying, watching files
Init container	runs first, then exits	waiting for a dependency, migrations, fetching config
Ambassador	local proxy to a remote service	hiding a database endpoint
Adapter	normalises the app's output	converting metrics or logs to a standard format

Init containers are the most common: they delay the application until it is safe to start.

output

```
    init cart downloader  -->  ready
             |
             v
    +-----------------+     +------------------+
    |  init migrator  | --> |  app container   |
    +-----------------+     +------------------+
```

2.3 Examples

Example 1: the smallest useful pod (not executed outputs).

YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: hello-pod
  labels:
    app: hello
spec:
  containers:
    - name: web
      image: nginx:1.27
      ports:
        - containerPort: 80
```

shell

```
kubectl apply -f pod.yaml
kubectl get pods -o wide
```

output

```
pod/hello-pod created

NAME        READY   STATUS    RESTARTS   AGE   IP            NODE
hello-pod   1/1     Running   0          12s   10.244.0.15   minikube
```

`1/1` means one of one containers is ready; the RESTARTS column tells you whether a container died and came back.

Example 2: a sidecar reading what the app writes.

YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: counted
spec:
  volumes:
    - name: shared
      emptyDir: {}
  containers:
    - name: writer
      image: busybox:1.36
      command: ["sh", "-c", "i=0; while true; do echo $i >> /shared/count; i=$((i+1)); sleep 5; done"]
      volumeMounts:
        - name: shared
          mountPath: /shared
    - name: reader
      image: busybox:1.36
      command: ["sh", "-c", "tail -f /shared/count"]
      volumeMounts:
        - name: shared
          mountPath: /shared
```

Both containers mount the same `emptyDir` volume, so the reader sees every line the writer appends. The shared volume lives as long as the pod does.

Example 3: wait for a dependency with an init container.

YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-with-init
spec:
  initContainers:
    - name: wait-for-db
      image: busybox:1.36
      command:
        - sh
        - -c
        - "until nc -z db 5432; do echo waiting for db; sleep 2; done"
  containers:
    - name: app
      image: myapp:1.4
      ports:
        - containerPort: 8080
```

The application container is not created until the init container succeeds.

Example 4: look inside a running pod (not executed).

shell

```shell
kubectl logs counted -c reader
kubectl exec -it counted -c writer -- sh
kubectl describe pod counted
```

output

```
0
1
2
(the exec command opens a shell inside the writer container)

Name:             counted
Namespace:        default
Priority:         0
Service Account:  default
Node:             minikube/192.168.49.2
Status:           Running
IP:               10.244.0.16
Init Containers:  <none>
Containers:
  writer:
    Image:          busybox:1.36
    State:          Running
  reader:
    Image:          busybox:1.36
    State:          Running
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  45s   default-scheduler  Successfully assigned default/counted to minikube
  Normal  Pulled     44s   kubelet            Container image "busybox:1.36" already present on machine
  Normal  Created    44s   kubelet            Created container writer
  Normal  Started    44s   kubelet            Started container writer
```

`describe` is the fastest debugging tool in Kubernetes: the events at the bottom explain every failure you will meet.

Example 5: a one-shot pod.

YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: batch-task
spec:
  restartPolicy: Never
  containers:
    - name: task
      image: busybox:1.36
      command: ["sh", "-c", "echo working; sleep 3; echo done"]
```

shell

```
kubectl get pod batch-task
```

output

```
NAME         READY   STATUS      RESTARTS   AGE
batch-task   0/1     Completed   0          15s
```

For anything that must run repeatedly or survive a failure, use a controller.

2.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Several unrelated processes in one container	slow builds, tangled failures	one process per container, sidecars for helpers
Assuming a restarted container keeps its filesystem	it starts from the image again	put shared state on a volume
Running a long-lived service as a bare pod	nothing recreates it after failure	deploy through a Deployment
Sharing a pod to make two apps "close"	they scale and fail together	give each its own Deployment and a Service
No requests on containers	the scheduler places pods blind	set requests on every container
`latest` as an image tag	no way to know what is running	pin a version or a digest
Running as root by default	privileges you never needed	set `runAsNonRoot` and drop capabilities

- If a pod is stuck in `Pending`, read the events: it is almost always scheduling or image pulls.
- Two containers in one pod share the IP, so they cannot use the same port.

2.5 Practice exercises

1. Two containers are in the same pod. Do they need a Service to talk to each other? Why?
2. Which container type must finish successfully before the app starts?
3. A pod shows `CrashLoopBackOff` in its status column. Which two commands do you run first?
4. Write the two lines of a pod spec that make the writer container of Example 2 mount the shared volume at `/shared`.
5. Why is `restartPolicy: Always` the default, and when is `Never` right?

Solutions

1. No. They share the network namespace, so one reaches the other on `localhost` and its port.
2. An init container, defined in `spec.initContainers`.
3. `kubectl logs <pod>` to see the crash output, and `kubectl describe pod <pod>` to read the restart events and exit codes.
4. `yaml volumeMounts: - name: shared mountPath: /shared` with the matching `volumes` entry in the pod spec.
5. Services should come back after a crash, so restarting inside the pod is the sane default. `Never` suits one-shot tasks where a retry must create a fresh pod, typically driven by a Job.

2.6 Mini test

1. What do all containers of one pod share? A) the image registry B) the network namespace and mounted volumes C) the same process tree D) nothing
2. Which phase means every container finished successfully? A) `Running` B) `Completed` C) `Succeeded` D) `Done`
3. Which field holds the list of containers? A) `spec.containers` B) `metadata.containers` C) `status.containers` D) `spec.pods`
4. A pod status shows `0/1` and `CrashLoopBackOff`. What is happening? A) the node is gone B) the container keeps exiting and being restarted C) the image is still pulling D) the pod finished
5. True or false: two containers in the same pod can both listen on port 80.
6. Which command streams the logs of one container in a multi-container pod? A) `kubectl logs pod -c container` B) `kubectl log pod/container` C) `kubectl exec pod -- logs` D) `kubectl describe pod`
7. Why is `emptyDir` called ephemeral? A) it is compressed B) it disappears when the pod is deleted C) it cannot be written D) it lives on etcd

Answer key and explanations

1. B - shared namespaces and volumes are what make a pod one scheduling unit.
2. C - the phase a job-style pod ends in after exit code 0.
3. A - containers (and init containers) live in the pod spec.
4. B - a crash loop means the process exits, the kubelet restarts it, and the cycle repeats.
5. False - they share one network namespace, so ports collide.
6. A - `-c` selects the container; without it you get the first one.
7. B - the volume is created with the pod and destroyed with it.

Chapter 3 - ReplicaSets and Deployments

3.1 Summary

- A ReplicaSet keeps a fixed number of identical pods running at all times.
- A Deployment manages ReplicaSets and is the object you actually use for stateless apps.
- The controller chain is Deployment to ReplicaSet to pods; each level watches the one above it.
- Label selectors bind a controller to its pods, and a name hash marks the pods it created.
- Pods created by a controller carry an owner reference, so deleting the owner deletes the pods.
- Scaling is a single number in the spec; self-healing is what happens when a pod disappears on its own.
- Rolling updates replace pods gradually, and the history you keep allows a rollback.
- `maxSurge` and `maxUnavailable` control how many pods may come and go during an update.

- A pod template hash in the ReplicaSet name makes each revision a distinct object.
- `kubectl rollout` is the vocabulary for status, history, pause, resume and undo.

3.2 Concepts

The ownership chain

Three objects, one responsibility each. The Deployment describes the app, the ReplicaSet delivers the count, the pods do the work.

output

```
      Deployment "web"                    replicas: 4
            |
            | creates and owns
            v
      ReplicaSet "web-7d9f8c4b"           replicas: 4
            |
            | creates and owns
            v
    +--------+--------+--------+--------+
    |        |        |        |        |
    v        v        v        v        v
  pod      pod      pod      pod      (one per replica)
```

Each ReplicaSet a Deployment creates is named after the pod template hash, so a template change produces a new ReplicaSet while the old one is scaled down.

What a ReplicaSet does

Behaviour	Detail
Count	creates pods until `replicas` matches `status.replicas`
Select	only adopts pods whose labels match `selector.matchLabels`
Own	sets `metadata.ownerReferences` on the pods it creates
Heal	replaces pods that are deleted or that lose their node
Ignore	leaves unrelated pods alone

ReplicaSets match by label, not by name, which is why two ReplicaSets with overlapping selectors in one namespace can fight over the same pods.

What a Deployment adds

Feature	How it shows up
Declarative updates	edit the template, apply, watch it roll out
Rollout history	old ReplicaSets are kept as revisions
`kubectl rollout undo`	swaps back to a previous ReplicaSet
Strategy control	`RollingUpdate` with `maxSurge` and `maxUnavailable`

Feature	How it shows up
Progress deadline	`progressDeadlineSeconds` marks a stuck rollout as failed
Pause and resume	`kubectl rollout pause` freezes the rollout mid-way

Selectors and templates

The selector lives above the template and the template must satisfy it, or the API server rejects the object.

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  labels:
    app: web
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx:1.27
```

That single file is the complete definition of a stateless application.

3.3 Examples

Example 1: a ReplicaSet on its own (not executed outputs).

YAML

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: queue-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      app: queue
  template:
    metadata:
      labels:
        app: queue
    spec:
      containers:
        - name: queue
          image: redis:7.2
```

shell

```
kubectl apply -f replicaset.yaml
kubectl get rs,pods
```

output

```
replicaset.apps/queue-rs created

NAME             DESIRED   CURRENT   READY   AGE
queue-rs         3         3         3       8s

NAME             READY   STATUS    RESTARTS   AGE
queue-rs-4kz2d   1/1     Running   0          8s
queue-rs-p9xwq   1/1     Running   0          8s
queue-rs-tt7v8   1/1     Running   0          8s
```

ReplicaSets are rarely written by hand today; Deployments supersede them.

Example 2: the Deployment you will write most often.

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  labels:
    app: web
spec:
  replicas: 3
  revisionHistoryLimit: 5
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: web
          image: nginx:1.27
          ports:
            - containerPort: 80
```

`maxSurge: 1` with `maxUnavailable: 0` lets one extra pod exist during the update, so capacity never drops.

Example 3: scale, watch and check ownership (not executed).

shell

```
kubectl scale deployment web --replicas=5
kubectl rollout status deployment web
kubectl get pods --show-labels
```

output

```
deployment.apps/web scaled

deployment "web" successfully rolled out

NAME                   READY   STATUS    RESTARTS   AGE   LABELS
web-7d9f8c4b-2l5nq     1/1     Running   0          40s   app=web,pod-template-hash=7d9f8c4b
web-7d9f8c4b-9v3mh     1/1     Running   0          40s   app=web,pod-template-hash=7d9f8c4b
web-7d9f8c4b-hd72k     1/1     Running   0          17s   app=web,pod-template-hash=7d9f8c4b
web-7d9f8c4b-m4tz8     1/1     Running   0          17s   app=web,pod-template-hash=7d9f8c4b
web-7d9f8c4b-qp6rw     1/1     Running   0          17s   app=web,pod-template-hash=7d9f8c4b
```

The `pod-template-hash` label is added by the controller and is how the ReplicaSet recognises its own pods.

Example 4: self-healing in action (not executed).

shell

```
kubectl delete pod web-7d9f8c4b-9v3mh
kubectl get pods -w
```

output

```
pod "web-7d9f8c4b-9v3mh" deleted

NAME                   READY   STATUS    RESTARTS   AGE
web-7d9f8c4b-2l5nq     1/1     Running   0          52s
web-7d9f8c4b-hd72k     1/1     Running   0          29s
web-7d9f8c4b-m4tz8     1/1     Running   0          29s
web-7d9f8c4b-qp6rw     1/1     Running   0          29s
web-7d9f8c4b-zz41c     1/1     Running   0          3s
```

The deleted pod is gone; a new one with a new name takes its place seconds later. That is the reconcile loop doing its job.

Example 5: read the revision history.

shell

```
kubectl rollout history deployment web
kubectl rollout history deployment web --revision=2
```

output

```
deployment.apps/web
REVISION  CHANGE-CAUSE
1         <none>
2         <none>

deployment.apps/web with revision #2
Pod Template:
  Labels:	app=web
	pod-template-hash=6b4f8d99c7
  Containers:
   web:
    Image:	nginx:1.28
```

Cause is empty unless you record it with `kubectl annotate deployment web kubernetes.io/change-cause="nginx 1.28"`.

3.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Overlapping selectors between two controllers	they fight over the same pods	one selector per app, per namespace
Editing the ReplicaSet instead of the Deployment	your change is overwritten	edit the Deployment
Immutable selector fields changed later	the Deployment stops matching itself	choose selectors carefully and never change them
Scaling a ReplicaSet a Deployment owns	the Deployment scales it back	scale the Deployment
`revisionHistoryLimit: 0`	no rollback targets	keep at least five revisions
Deploying with `--record` expectations	it is deprecated	annotate the change cause instead
One Deployment per microservice with no labels	operators cannot see the app	add `app`, `tier` and `release` labels

- `kubectl get deploy` is the health check of an application: READY column should match DESIRED.
- Deleting a Deployment deletes its ReplicaSets and pods; add `--cascade=orphan` only when you mean it.

3.5 Practice exercises

1. Draw the ownership chain from Deployment to the running containers, naming each object.
2. Which two blocks must agree inside a Deployment, and what does the API do if they do not?
3. A Deployment has `replicas: 3` and one pod is deleted. What do you observe in the next minute?
4. Write the `strategy` block that allows at most one extra pod and zero unavailable pods during an update.
5. What does the `pod-template-hash` label tell you?

Solutions

1. Deployment to ReplicaSet to pods; the ReplicaSet controller creates the pods, the scheduler assigns each one to a node, and the kubelet on that node runs the containers. Owner references run down the chain so deleting the Deployment removes everything below it.
2. `spec.selector.matchLabels` must be satisfied by `spec.template.metadata.labels`; otherwise the API server rejects the Deployment with a validation error.

3. The ReplicaSet controller notices one pod missing and immediately creates a replacement with a new name; `kubectl rollout status` continues to report the deployment as complete once all replicas are ready again.
4. `yaml strategy: type: RollingUpdate rollingUpdate: maxSurge: 1 maxUnavailable: 0`
5. That the pod belongs to a specific ReplicaSet revision, so different template versions can coexist during a rollout.

3.6 Mini test

1. Which object creates pods in a healthy Deployment? A) the Deployment controller, directly B) the ReplicaSet C) the scheduler D) kubelet
2. What matches a ReplicaSet to its pods? A) the pod name pattern B) label selectors C) the namespace alone D) owner references alone
3. Which command rolls an application back to the previous revision? A) `kubectl undo deployment web` B) `kubectl rollout undo deployment web` C) `kubectl rollback web` D) `kubectl revert deployment/web`
4. True or false: a Deployment owns its pods through owner references.
5. What does `maxUnavailable: 0` guarantee during a rolling update? A) the update is instant B) no pod is ever taken down before a replacement is ready C) pods are never recreated D) the rollout pauses automatically
6. Which value keeps five old ReplicaSets for rollback? A) `maxSurge: 5` B) `replicas: 5` C) `revisionHistoryLimit: 5` D) `progressDeadlineSeconds: 5`
7. A ReplicaSet shows DESIRED 3, CURRENT 2. What happens next? A) nothing, the controller waits for you B) the controller creates one more pod C) the ReplicaSet is deleted D) the node is drained

Answer key and explanations

1. B - the Deployment controller manages a ReplicaSet, and the ReplicaSet controller creates pods.
2. B - selectors are the loose coupling that lets controllers find work.
3. B - `kubectl rollout undo` returns to a previous template revision.
4. True - owner references form the deletion chain.
5. B - with no unavailable slots the new pod must turn ready before an old one is removed.
6. C - that field is the size of the revision history that is kept.
7. B - count and act: one pod is created to close the gap.

Chapter 4 - Services and cluster DNS

4.1 Summary

- A Service gives a changing set of pods one stable name and one virtual IP.

- The Service selector finds pods, and the endpoints object tracks their current addresses.
- `kube-proxy` on every node programs the packet rules that make the virtual IP work.
- `ClusterIP` is the default and is reachable only inside the cluster.
- `NodePort` opens a port on every node, and `LoadBalancer` asks the cloud for an external address.
- A headless Service (`clusterIP: None`) returns pod addresses instead of a virtual IP.
- `ExternalName` maps a Service name to a DNS name outside the cluster.
- Every Service gets a DNS name from CoreDNS: `name.namespace.svc.cluster.local`.
- Services never proxy traffic; they are virtual IPs plus rules, and the pods answer directly.
- Ports have two names: `port` for the Service side and `targetPort` for the container side.

4.2 Concepts

Why Services exist

Pods come and go, and their IPs change with them. A Service is a stable front door: a constant name, a constant virtual IP, and a controller that keeps the list of endpoints up to date.

output

```
client pod                        Service "web"
+-----------+     web.default     +-------------------+
|   curl    | ------------------> | 10.96.12.47:80    |
+-----------+                     | selector app=web  |
                                  +---------+---------+
                                            |
                                  keeps endpoints in sync
                                            |
                     +----------------------+--------------------------+
                     |                      |                          |
                     v                      v                          v
               +------------+           +------------+           +------------+
               | 10.244.0.9 |           | 10.244.1.3 |           | 10.244.2.7 |
               |  pod:8080  |           |  pod:8080  |           |  pod:8080  |
               +------------+           +------------+           +------------+
```

The `web` name above is short because the client is in the `default` namespace; from another namespace the name is `web.default`.

The four Service types

Type	Reachable from	Port	Use
`ClusterIP`	inside the cluster	a virtual IP in the service range	default choice for internal apps
`NodePort`	`nodeIP:port` from outside	the ClusterIP port plus ~30000-32767	experiments, bare metal
`LoadBalancer`	the internet	provisions a cloud load balancer	public entry point

Type	Reachable from	Port	Use
`ExternalName`	inside the cluster	no IP at all	DNS alias to an external service

A `NodePort` or `LoadBalancer` Service also has a ClusterIP, so cluster clients use the same name.

Ports and targetPort

YAML

```yaml
ports:
  - name: http
    port: 80          # what clients connect to
    targetPort: 8080   # the port on the pod
    protocol: TCP
```

When `targetPort` is omitted it defaults to `port`. Naming ports matters when one Service exposes more than one port, because each name must be unique.

Endpoints

A Service builds an Endpoints object (or EndpointSlice) with one entry per matching pod address. If the list is empty, connections to the Service fail, which is the usual symptom of a label mismatch.

output

```
$ kubectl get endpoints web
NAME   ENDPOINTS                       AGE
web    10.244.0.9:8080,10.244.1.3:8080,10.244.2.7:8080   2m
```

Cluster DNS

CoreDNS watches Services and writes a record per Service:

Record	Example
`service`	`web.default.svc.cluster.local`
`service.namespace`	`web.default`
`service.namespace.svc`	`web.default.svc`
pod-backed A record (headless)	`web-0.web.default.svc.cluster.local`

The `cluster.local` suffix is configurable at install time; the search path inside pods usually makes the short name work.

4.3 Examples

Example 1: expose a Deployment inside the cluster.

YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  type: ClusterIP
  selector:
    app: web
  ports:
    - name: http
      port: 80
      targetPort: 8080
      protocol: TCP
```

shell

```shell
kubectl apply -f service.yaml
kubectl get svc,endpoints web
```

output

```
service/web created

NAME   TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE
web    ClusterIP   10.96.12.47    <none>        80/TCP    5s

NAME   ENDPOINTS                                         AGE
web    10.244.0.9:8080,10.244.1.3:8080,10.244.2.7:8080   5s
```

Example 2: call the Service by name from another pod (not executed).

shell

```shell
kubectl run client --rm -it --image=busybox:1.36 --restart=Never -- sh
wget -qO- http://web.default.svc.cluster.local/
```

output

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
</head>
...
```

The short name `http://web/` works too, because the resolver appends the search path of the namespace.

Example 3: expose on a node port.

YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-nodeport
spec:
  type: NodePort
  selector:
    app: web
  ports:
    - name: http
      port: 80
      targetPort: 8080
      nodePort: 30080
```

shell

```
minikube service web-nodeport --url
```

output

```
http://192.168.49.2:30080
```

Without an explicit `nodePort`, the API server picks a free one in the 30000-32767 range.

Example 4: a headless Service for direct pod addressing.

YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-headless
spec:
  clusterIP: None
  selector:
    app: web
  ports:
    - name: http
      port: 80
      targetPort: 8080
```

shell

```
kubectl get endpointslices -l kubernetes.io/service-name=web-headless
```

output

```
NAME                  ADDRESSTYPE   PORTS   ENDPOINTS          AGE
web-headless-7xk2m    IPv4          8080    10.244.0.9,10.244.1.3,10.244.2.7   30s
```

StatefulSets and client-side load balancing need this: DNS returns the pod addresses themselves.

Example 5: an external alias.

YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: db
spec:
  type: ExternalName
  externalName: db.internal.example.com
```

shell

```
kubectl exec client -- nslookup db.default.svc.cluster.local
```

output

```
Server:  10.96.0.10
Address: 10.96.0.10#53

db.default.svc.cluster.local canonical name = db.internal.example.com.
Name: db.internal.example.com
Address: 203.0.113.24
```

No selector, no endpoints, no proxying: DNS answers with the target name.

4.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Service selector that matches no pod	connections time out with empty endpoints	keep `kubectl get endpoints` in your checks
`targetPort` pointing at the wrong container port	traffic never arrives	match the `containerPort` exactly
Using `NodePort` in production	random ports and no TLS	put an Ingress or LoadBalancer in front
Creating a Service before the pods	empty endpoints at first	apply the workload first, or accept a short warm-up
Two port entries with the same name	the API server rejects the object	unique names when `name` is set
Publishing a Database with `LoadBalancer`	public exposure of state	keep state on ClusterIP, expose it through the app

- When a Service misbehaves, compare `kubectl describe svc` with the pod labels.
- `kubectl port-forward svc/web 8080:80` tunnels a Service to your laptop without any node port.

4.5 Practice exercises

1. A Service has a virtual IP but no endpoints. Name the most likely cause and the command that proves it.

2. Write a Service manifest that sends port 80 on the Service to container port 3000 on pods labelled `app: api`.
3. Which Service type makes DNS return pod addresses instead of one virtual IP?
4. What is the full DNS name of the Service `orders` in namespace `shop`?
5. Why is `NodePort` rarely the right public entry point?

Solutions

1. The selector matches no pod, or the pods have no ready container. Compare `kubectl describe svc NAME` with `kubectl get pods --show-labels`; the endpoints list proves the mismatch.
2. `yaml apiVersion: v1 kind: Service metadata: name: api spec: selector: app: api ports: - name: http port: 80 targetPort: 3000`
3. A headless Service, `clusterIP: None`.
4. `orders.shop.svc.cluster.local`.
5. It exposes a high random port on every node with no TLS, no host routing and no shared entry point; an Ingress or a cloud load balancer does that job.

4.6 Mini test

1. What does a Service provide that pods do not? A) a stable name and virtual IP B) persistent storage C) a container image D) a node to run on
2. Which object holds the list of pod addresses for a Service? A) Endpoints or EndpointSlice B) ConfigMap C) ReplicaSet D) Ingress
3. Which field of a Service points at the container port? A) `port` B) `nodePort` C) `targetPort` D) `clusterIP`
4. Which Service type returns pod addresses through DNS? A) `ClusterIP` B) `NodePort` C) headless (`clusterIP: None`) D) `LoadBalancer`
5. True or false: a Service proxies every request it forwards.
6. Which component programs the rules that implement Service virtual IPs? A) CoreDNS B) kube-proxy C) etcd D) the scheduler
7. What is the DNS short name of Service `web` in namespace `shop`, used from a pod in the same namespace? A) `web` B) `web.shop` C) `web.svc.cluster.local` D) `cluster.local/web`

Answer key and explanations

1. A - the name and the virtual IP survive every pod restart.
2. A - the endpoints controller watches pods and keeps the list current.
3. C - `targetPort` is the container side, `port` is the client side.
4. C - headless Services hand out addresses instead of a cluster IP.
5. False - the pods answer directly; the Service only sets up addressing.
6. B - kube-proxy writes the packet rules on each node.
7. A - the namespace search path in the pod completes the name.

Chapter 5 - ConfigMaps and Secrets

5.1 Summary

- A ConfigMap holds non-confidential configuration; a Secret holds anything that must not be readable by everyone.
- Both can be injected as environment variables, command arguments, or mounted files.
- Mounted files update automatically when the object changes; environment variables never do.
- Secrets are base64 encoded, not encrypted: anyone with read access to the namespace can decode them.
- `stringData` accepts plain values and stores them encoded; `data` expects values already encoded.
- Common Secret types are `Opaque`, `kubernetes.io/tls`, `kubernetes.io/dockerconfigjson` and `kubernetes.io/basic-auth`.
- `immutable: true` on a ConfigMap or Secret protects it from accidental writes and lowers API load.
- Configuration should travel with the deployment manifest, never with the image.
- Secret access is controlled by RBAC, and encryption at rest is an etcd-level setting of the cluster.
- Volume-mounted values only appear inside the container's filesystem when the mount path is correct.

5.2 Concepts

Where configuration belongs

Twelve-factor applications read configuration from the environment. Kubernetes offers two objects for that, with the same consumption patterns:

	ConfigMap	Secret
Holds	configuration, feature flags, file fragments	passwords, tokens, keys, certificates
Size limit	about 1 MiB	about 1 MiB
Encoding	plain text	base64 in `data`, plain in `stringData`
Read control	RBAC like any object	RBAC like any object, but plan tighter rules
Updates	visible to mounted files	visible to mounted files, needs pod restart for env

Three ways to consume values

output

```
                     +----------------------+
                     | ConfigMap / Secret   |
                     +----------+-----------+
                                |
           +--------------------+---------------------+
           |                    |                     |
           v                    v                     v
    envFrom: all keys     valueFrom: one key      volume: files on disk
    as environment        as one environment       mounted at a path
    variables             variable
           |                    |                     |
           v                    v                     v
    app start              app start             read at any time,
    snapshot               snapshot               live updates
```

Environment variables are read once at process start; mounted files are served from a projected volume the kubelet keeps in sync, which is why config servers and template reloaders prefer mounts.

Volumes are files, not values

A ConfigMap becomes a directory: one key per file.

output

```
container
  /etc/app/config
    database.url      <- key "database.url"
    feature.flags     <- key "feature.flags"
```

`subPath` mounts a single key as one file, and `items` can remap or filter keys inside the mount.

5.3 Examples

Example 1: a ConfigMap and a pod that reads it.

YAML

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  database.url: "postgres://db.default.svc:5432/shop"
  log.level: "info"
  greeting: |
    Hello from the ConfigMap.
    Line two.
```

YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: app
spec:
  containers:
    - name: app
      image: myapp:1.0
      env:
        - name: DATABASE_URL
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: database.url
        - name: LOG_LEVEL
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: log.level
      volumeMounts:
        - name: config
          mountPath: /etc/app
          readOnly: true
  volumes:
    - name: config
      configMap:
        name: app-config
```

Inside the container, `env | grep -E "DATABASE|LOG"` shows the two values, and `ls /etc/app` shows three files, one per key.

Example 2: create a ConfigMap from the command line (not executed).

shell

```shell
kubectl create configmap app-config \
  --from-literal=database.url=postgres://db.default.svc:5432/shop \
  --from-literal=log.level=info \
  --from-file=./greeting.txt \
  --dry-run=client -o yaml > app-config.yaml
```

output

```yaml
apiVersion: v1
data:
  database.url: postgres://db.default.svc:5432/shop
  greeting.txt: |
    Hello from the file.
  log.level: info
kind: ConfigMap
metadata:
  creationTimestamp: null
  name: app-config
```

The `--dry-run=client -o yaml` pattern turns a quick command into a manifest you can keep in Git.

Example 3: a Secret with `stringData`, and the same object as stored.

YAML

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
stringData:
  username: shop
  password: "s3cr3t-value"
```

shell

```shell
kubectl get secret db-credentials -o jsonpath='{.data}'
```

output

```
{"password":"czNjcjN0LXZhbHVl","username":"c2hvcA=="}
```

`stringData` is write-only sugar: the API server encodes it and the returned object shows the encoded `data` form.

Example 4: mount a TLS secret into a web server.

YAML

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: web-tls
type: kubernetes.io/tls
data:
  tls.crt: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCg==
  tls.key: LS0tLS1CRUdJTiBSU0EUFUt...
```

YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: tls-web
spec:
  containers:
    - name: web
      image: nginx:1.27
      volumeMounts:
        - name: certs
          mountPath: /etc/tls
          readOnly: true
  volumes:
    - name: certs
      secret:
        secretName: web-tls
```

The `kubernetes.io/tls` type requires exactly `tls.crt` and `tls.key`; other keys are rejected.

Example 5: immutable objects.

YAML

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: pinned-config
immutable: true
data:
  log.level: "warn"
```

An immutable object cannot be changed, only replaced with a new name. Use it for configuration that belongs to a release, and remember to update the pod reference in the same deployment.

5.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Baking configuration into the image	one image per environment	one image, one ConfigMap per environment
Expecting env vars to refresh	the process never sees the change	mount as files, or restart the pods
Committing secrets to Git in plain text	everyone with repo access has them	encrypt at rest, seal the values, use an external store
Base64 called "encryption"	decoding is one command away	protect with RBAC and plan etcd encryption
Missing ConfigMap key	the pod stays in `CreateContainerConfigError`	`kubectl describe pod` and fix the key name
`subPath` used everywhere	updates inside the directory never arrive	mount the whole directory when you need live updates
Wrong mount path	the application reads empty or default values	verify with `kubectl exec -- ls`

- Roll a new configuration the same way you roll an image: change the object, then restart or redeploy the workload.
- Keep secrets out of `envFrom` when only one value is needed; smaller grants are easier to audit.

5.5 Practice exercises

1. A deployment references ConfigMap key `log.level` but the pod never starts and the events mention a missing key. What do you check?
2. Write a Secret manifest with type `Opaque` setting `username` to `admin` and `password` to `p@ss`, using `stringData`.
3. Why do mounted ConfigMap files update while injected environment variables do not?
4. Which command prints the base64 encoding of a password before you paste it into a Secret?

5. What does `immutable: true` cost you, and when is it worth it?

Solutions

1. `kubectl describe pod <name>` for the event, then `kubectl get configmap <name> -o yaml` to compare the exact key names; ConfigMap keys are case sensitive.
2. `yaml apiVersion: v1 kind: Secret metadata: name: admin-credentials type: Opaque stringData: username: admin password: "p@ss"`
3. Mounted files come from a projected volume the kubelet refreshes, while environment variables are captured when the process starts.
4. `echo -n 'p@ss' | base64` (or `[Convert]::ToBase64String([Text.Encoding]::UTF8.GetBytes('p@ss'))` on PowerShell).
5. You lose the ability to edit the object in place: every change needs a new name and a workload reference update. It is worth it for values tied to one release, because it prevents accidental edits and cuts API server load.

5.6 Mini test

1. Which object is designed for connection strings and feature flags? A) Secret B) ConfigMap C) PersistentVolume D) Namespace
2. What does base64 give a Secret? A) encryption at rest B) transport-safe encoding only C) a password manager D) RBAC enforcement
3. Which injection method lets the container see updates without a restart? A) `env` with `valueFrom` B) `envFrom` C) a volume mount D) `command` arguments
4. Which Secret type requires the keys `tls.crt` and `tls.key`? A) `Opaque` B) `kubernetes.io/basic-auth` C) `kubernetes.io/tls` D) `kubernetes.io/service-account-token`
5. True or false: a ConfigMap key can be mounted as a single file with `subPath`, and that file still updates when the ConfigMap changes.
6. Which field accepts plain-text values for a Secret? A) `data` B) `stringData` C) `binaryData` D) `values`
7. A pod references a ConfigMap that does not exist. What happens? A) the pod runs with empty values B) the pod fails to start until the ConfigMap exists C) the node reboots D) the Service loses its endpoints

Answer key and explanations

1. B - non-confidential configuration belongs in a ConfigMap.
2. B - encoding is not encryption; anyone who can read the object can decode it.
3. C - projected volumes are refreshed by the kubelet.
4. C - the TLS type is validated for those exact keys.
5. False - `subPath` mounts a snapshot path, so updates are not seen.
6. B - `stringData` is write-only and gets encoded on the server.
7. B - volume and env references are resolved before the container starts.

Chapter 6 - Storage: volumes, PVs, PVCs and StatefulSets

6.1 Summary

- Volumes live and die with the pod; persistent volumes outlive them.
- `emptyDir` is scratch space shared by the containers of one pod.
- A PersistentVolume (PV) is a piece of storage in the cluster; a PersistentVolumeClaim (PVC) is a request for it.
- The control plane binds a PVC to a matching PV, and the pod mounts the claim, not the storage itself.
- Access modes limit how a volume may be attached: read-write-once, read-only-many, read-write-many.
- A StorageClass with `volumeBindingMode: WaitForFirstConsumer` delays provisioning until the pod's node is known.
- `reclaimPolicy` decides whether a released PV is kept or deleted.
- StatefulSets give pods stable names and stable storage, which databases need.
- `volumeClaimTemplates` creates one claim per replica, and the claim survives pod deletion.
- Storage is the usual reason a workload cannot simply be deleted and recreated.

6.2 Concepts

The storage chain

Pods never name storage directly. They ask for a claim, the claim is bound to a volume, and the volume points at the real backend.

output

```
       pod                          PersistentVolumeClaim          PersistentVolume
+---------------+              +---------------------+          +------------------+
|  volumeMounts |------------->| web-data            |--------->| pvc-8f3a1c       |
|  /var/lib/data|              | requests: 5Gi       |  BOUND   | capacity: 5Gi    |
+---------------+              | accessModes: [RWO]  |          | hostPath or NFS  |
                               +---------------------+          +------------------+
```

The pod's spec is portable: the same manifest works on a developer laptop and in production because only the claim is referenced.

Volume kinds you will meet

Kind	Lifetime	Shared between	Typical use
`emptyDir`	the pod	containers of one pod	scratch, sidecar hand-off
`configMap` / `secret`	the object	one pod	configuration files

Kind	Lifetime	Shared between	Typical use
`hostPath`	the node	pods on that node	node agents, rarely apps
PersistentVolumeClaim	the claim	up to the access mode	databases, uploads

Access modes and policies

Mode	Meaning
`ReadWriteOnce`	one node can mount it read-write
`ReadOnlyMany`	many nodes can mount read-only
`ReadWriteMany`	many nodes can mount read-write
`ReadWriteOncePod`	exactly one pod, in the whole cluster

Storage classes add three knobs: `provisioner` (which backend), `reclaimPolicy` (`Delete` or `Retain`) and `volumeBindingMode` (`Immediate` or `WaitForFirstConsumer`). Local disk and zonal cloud disks need the delayed binding to avoid scheduling a pod next to storage it cannot reach.

PV and PVC matching

A claim is bound when a PV matches its requests. A PV can carry its own claim reference, which pins a specific claim to a specific volume.

YAML

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: manual-pv
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual
  claimRef:
    apiVersion: v1
    kind: PersistentVolumeClaim
    namespace: default
    name: manual-pvc
  hostPath:
    path: /data/manual-pv
```

StatefulSets

A StatefulSet is a Deployment with identity. Pods get ordinal names, are created in order, and keep their storage across restarts.

output

```
   StatefulSet "db"  replicas: 3
          |
          +--> db-0  <-> pvc db-0  (kept on deletion)
          +--> db-1  <-> pvc db-1
          +--> db-2  <-> pvc db-2
```

A stable network identity comes from a headless Service named in `serviceName`, so `db-0.db` resolves to that pod. Writing to the ordinal address is the standard pattern for leader election.

6.3 Examples

Example 1: scratch space shared inside a pod.

YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: scratch
spec:
  containers:
    - name: writer
      image: busybox:1.36
      command: ["sh", "-c", "echo payload > /scratch/payload; sleep 3600"]
      volumeMounts:
        - name: scratch
          mountPath: /scratch
  volumes:
    - name: scratch
      emptyDir:
        sizeLimit: 64Mi
```

`sizeLimit` is a guard, not a quota: the writer is evicted when it exceeds the limit instead of filling the node's disk.

Example 2: a manual PV and PVC pair.

YAML

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: manual-pv
spec:
  capacity:
    storage: 2Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual
  hostPath:
    path: /data/manual-pv
```

YAML

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: manual-pvc
spec:
  storageClassName: manual
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 2Gi
```

YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: with-pvc
spec:
  containers:
    - name: app
      image: nginx:1.27
      volumeMounts:
        - name: data
          mountPath: /usr/share/nginx/html
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: manual-pvc
```

shell

```
kubectl get pv,pvc
```

output

```
NAME                        CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM
manual-pv                   2Gi        RWO            Retain           Bound    default/manual-pvc

NAME          STATUS   VOLUME       CAPACITY   ACCESS MODES   STORAGECLASS   AGE
manual-pvc    Bound    manual-pv    2Gi        RWO            manual         5s
```

`hostPath` works on a single-node cluster only; a real cluster provisions storage from the cloud or an NFS server.

Example 3: dynamic provisioning with the default class.

YAML

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: cache
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
```

Omitting `storageClassName` uses the cluster's default class, which on minikube is a `hostpath` provisioner that creates volumes under `/data` automatically.

Example 4: a StatefulSet with one volume per replica.

YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: db
spec:
  clusterIP: None
  selector:
    app: db
  ports:
    - name: db
      port: 5432
```

YAML

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: db
spec:
  serviceName: db
  replicas: 3
  selector:
    matchLabels:
      app: db
  template:
    metadata:
      labels:
        app: db
    spec:
      containers:
        - name: postgres
          image: postgres:16
          ports:
            - containerPort: 5432
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes:
          - ReadWriteOnce
        resources:
          requests:
            storage: 10Gi
```

Deleting pod `db-1` reattaches the same claim `data-db-1` to the replacement pod, so the data survives.

Example 5: reclaim policy in practice (not executed).

shell

```
kubectl delete pvc manual-pvc
kubectl get pv manual-pv
```

output

```
persistentvolumeclaim "manual-pvc" deleted

NAME        CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM
manual-pv   2Gi        RWO            Retain           Released  default/manual-pvc
```

With `Retain` the data stays on disk and the PV waits for an administrator; `Delete` erases the backend when the claim disappears.

6.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`ReadWriteMany` on a block volume	the claim never binds	check the provisioner's supported modes
Deleting a PVC to "fix" a mount	data loss unless `Retain`	back up first, check the reclaim policy
`hostPath` in production	ties pods to one node	use a StorageClass provisioned backend
Ignoring `volumeBindingMode`	pods scheduled where storage is unreachable	use `WaitForFirstConsumer` for zonal storage
Stateful data on `emptyDir`	silently lost on restart	use a PVC, ideally via `volumeClaimTemplates`
One shared PVC for a multi-replica RWO app	pods block each other	one claim per replica, or a RWX backend

- `kubectl describe pvc` names the PV it bound to and the events that delayed the binding.
- Deployments can be rescheduled anywhere; only StatefulSets (or controllers with stable identity) keep a storage identity.

6.5 Practice exercises

1. A PVC stays `Pending`. Give three possible reasons.
2. Which object does the pod reference: the PV, the PVC, or the StorageClass?
3. Write the `volumes` entry that mounts a PVC named `assets` at `/data`.
4. What does `persistentVolumeReclaimPolicy: Retain` do, and when would you choose `Delete`?
5. Why does `db-1` keep its data when the pod is recreated in a StatefulSet?

Solutions

1. No matching PV exists and the class cannot provision; a PV exists but its access modes or size do not match; the class uses `WaitForFirstConsumer` and no pod consumes the claim yet.

2. The PVC; the binding to a PV is the cluster's job.
3. `yaml volumes: - name: assets persistentVolumeClaim: claimName: assets`
4. It keeps the PV and its data after the claim is deleted, marking it `Released` for manual cleanup. Choose `Delete` when the backend may be discarded with the claim, typically for caches and throwaway volumes.
5. Each replica owns a claim created from `volumeClaimTemplates` (`data-db-1`), and claims are not deleted when their pod is removed, so the replacement pod binds the same storage.

6.6 Mini test

1. Which object does a pod mount to get persistent storage? A) a PersistentVolume B) a PersistentVolumeClaim C) a StorageClass D) an Endpoints object
2. What is the lifetime of an `emptyDir` volume? A) forever B) the life of the pod C) the life of the node D) the life of the namespace
3. Which access mode allows many nodes to write at once? A) `ReadWriteOnce` B) `ReadOnlyMany` C) `ReadWriteMany` D) `ReadWriteOncePod`
4. Which StatefulSet field creates one PVC per replica? A) `volumeMounts` B) `volumes` C) `volumeClaimTemplates` D) `claims`
5. True or false: deleting a PVC with a `Delete` reclaim policy provisioner keeps the data on the backend.
6. What does `clusterIP: None` do to a Service? A) makes it public B) makes it headless: DNS returns pod addresses C) removes its DNS name D) blocks all traffic
7. Which `volumeBindingMode` is wrong for zonal disks with delayed scheduling? A) `WaitForFirstConsumer` B) `Immediate` C) `Manual` D) `Never`

Answer key and explanations

1. B - the claim is the stable, portable interface between pod and storage.
2. B - `emptyDir` is created with the pod and deleted with it.
3. C - `ReadWriteMany` is the multi-writer mode, usually backed by a file service.
4. C - the templates are stamped per replica with the stateful name.
5. False - `Delete` removes the backend; `Retain` keeps it.
6. B - headless Services are the addressing scheme behind StatefulSets.
7. B - immediate binding provisions storage before the pod's node is known, which can strand the volume in another zone.

Chapter 7 - Probes, requests and limits

7.1 Summary

- A probe is a health check the kubelet runs against a container on a schedule.
- `startupProbe` protects slow-starting applications from being killed too early.

- `readinessProbe` decides whether a Service sends traffic to the pod.
- `livenessProbe` decides whether a stuck container is restarted.
- Resource `requests` influence scheduling; `limits` influence what the runtime enforces.
- Requests and limits are per container, not per pod, and CPU limits cause throttling rather than eviction.
- Memory is incompressible: exceeding a memory limit means an out-of-memory kill.
- The three QoS classes are Guaranteed, Burstable and BestEffort, decided by the requests and limits.
- `ResourceQuota` caps the total a namespace may consume; `LimitRange` sets per-object defaults.
- `kubectl top` shows live usage, but only when the metrics add-on is installed.

7.2 Concepts

The three probes

output

```
container start
      |
      v
+-----------+     fails: restart the container
|  startup  |----------------------------> (kubelet retries)
+-----------+
      | success
      v
+-----------+     fails: pod leaves the Service endpoints
| readiness |----------------------------> (no traffic)
+-----------+
      |                                       ^
      v                                       |
+-----------+     fails: kill and restart    |
| liveness  |--------------------------------+
+-----------+
      |
      v
steady state
```

Probe	Question it answers	Failure effect
startup	is the application still booting?	container is restarted, repeatedly
readiness	can this pod take traffic?	removed from Service endpoints
liveness	is this container alive enough to continue?	container is killed and restarted

A probe handler is one of `httpGet`, `tcpSocket`, `exec` or `grpc`:

YAML

```yaml
readinessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10
  timeoutSeconds: 2
  successThreshold: 1
  failureThreshold: 3
```

Startup probes exist because `initialDelaySeconds` is a guess; a probe that waits for the app is a probe that never kills a slow but healthy container.

Requests and limits

YAML

```yaml
resources:
  requests:
    memory: "128Mi"
    cpu: "250m"
  limits:
    memory: "256Mi"
    cpu: "500m"
```

Field	Used by	Effect
`requests.cpu`	scheduler	fraction of a core reserved; scheduling input only
`requests.memory`	scheduler	memory reserved; also an eviction signal
`limits.cpu`	runtime	the container is throttled after it uses this much
`limits.memory`	runtime	the container is OOM-killed after it uses this much

CPU is measured in millicores: `250m` is a quarter of a core. Memory uses `Mi` and `Gi`, powers of two, which is what the scheduler understands.

QoS classes

Class	Condition	Eviction order
Guaranteed	every container has requests equal to limits	last
Burstable	at least one request is set, some are not equal to limits	middle
BestEffort	no requests or limits at all	first

The class is decided from the pod, not the namespace, and it is what the kubelet uses when the node runs out of memory.

Quota and defaults

YAML

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: team-quota
spec:
  hard:
    requests.cpu: "4"
    requests.memory: 8Gi
    limits.cpu: "8"
    limits.memory: 16Gi
    count/deployments.apps: "10"
```

YAML

```yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
spec:
  limits:
    - type: Container
      default:
        cpu: "500m"
        memory: "256Mi"
      defaultRequest:
        cpu: "100m"
        memory: "128Mi"
```

With a quota active, a pod without requests is rejected: namespaces should carry a LimitRange so that defaults exist.

7.3 Examples

Example 1: a production-shaped container.

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: api
  template:
    metadata:
      labels:
        app: api
    spec:
      containers:
        - name: api
          image: myapi:2.1
          ports:
            - containerPort: 8080
          startupProbe:
            httpGet:
              path: /startup
              port: 8080
            failureThreshold: 30
            periodSeconds: 2
          readinessProbe:
            httpGet:
              path: /ready
              port: 8080
            periodSeconds: 5
          livenessProbe:
            httpGet:
              path: /live
              port: 8080
            periodSeconds: 10
          resources:
            requests:
              cpu: "250m"
              memory: "256Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
```

The startup probe allows up to a minute of boot time; readiness and liveness start only after it succeeds.

Example 2: watch probes work (not executed).

shell

```
kubectl port-forward deploy/api 8080:8080
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8080/ready
```

output

```
200
```

shell

```
kubectl get pods -w
```

output

```
NAME                   READY   STATUS    RESTARTS   AGE
api-6d4f8c9b-7xk2m     0/1     Running   0          4s
api-6d4f8c9b-7xk2m     1/1     Running   0          9s
api-6d4f8c9b-m4tz8     0/1     Running   0          4s
api-6d4f8c9b-m4tz8     1/1     Running   0          9s
api-6d4f8c9b-zz41c     0/1     Running   0          4s
api-6d4f8c9b-zz41c     1/1     Running   0          9s
```

The `0/1` phase is the readiness probe failing while the app warms up.

Example 3: catch a failing liveness probe (not executed).

shell

```
kubectl describe pod api-6d4f8c9b-7xk2m
```

output

```
Containers:
  api:
    State:          Running
    Ready:          False
    Restart Count:   2
    Liveness:       http-get http://:8080/live delay=0s timeout=1s period=10s #success=1 #failure=3
Events:
  Type     Reason     Age                From               Message
  ----     ------     ----               ----               -------
  Normal   Scheduled  3m                 default-scheduler  Successfully assigned default/api-6d4f8c9b-
  Normal   Pulled     3m                 kubelet            Container image "myapi:2.1" already present
  Normal   Created    3m                 kubelet            Created container api
  Warning  Unhealthy  2m (x3 over 100s)  kubelet            Liveness probe failed: HTTP probe failed wi
  Normal   Killing    2m                 kubelet            Container api failed liveness probe, will b
```

`Restart Count` combined with the events is the whole story: which probe failed, with what result, and how often.

Example 4: live usage from the metrics add-on (not executed).

shell

```
kubectl top pods
kubectl top nodes
```

output

```
NAME                       CPU(cores)   MEMORY(bytes)
api-6d4f8c9b-7xk2m         12m          64Mi
api-6d4f8c9b-m4tz8         15m          67Mi
api-6d4f8c9b-zz41c         11m          63Mi

NAME       CPU(cores)   CPU%   MEMORY(bytes)   MEMORY%
minikube   214m         10%    1420Mi          37%
```

`kubectl top` reads from the metrics API. Without an add-on such as metrics-server the command fails with a clear error instead of returning zeros.

Example 5: a LimitRange that keeps a namespace honest.

YAML

```yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: mem-cpu-defaults
  namespace: study
spec:
  limits:
    - type: Container
      max:
        cpu: "2"
        memory: 1Gi
      default:
        cpu: "500m"
        memory: "256Mi"
      defaultRequest:
        cpu: "100m"
        memory: "128Mi"
```

Any container in `study` that asks for more than the maximum is rejected, and any container without requests silently receives the defaults.

7.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Liveness probe checks dependencies	one broken database restarts every pod	probe the container only
No startup probe for a slow app	the liveness probe kills it mid-boot	cover boot time with `startupProbe`
Readiness and liveness hitting the same endpoint	a dead pod keeps serving traffic	separate deep and shallow endpoints
Requests far below real usage	the node is oversubscribed	measure with `kubectl top` and adjust
Memory limits below the real peak	OOM kills under load	leave headroom for spikes
Only limits, no requests	scheduling ignores the pod	always set requests, treat limits as ceilings
Quota without a LimitRange	every pod without requests is rejected	install both together

- `failureThreshold` times `periodSeconds` is the total time a container gets to fail before the kubelet acts: at least 30 seconds for liveness.
- CPU throttling is invisible in `kubectl top`; latency graphs reveal it.

7.5 Practice exercises

1. Which probe controls Service endpoints, and which one restarts containers?
2. A Spring-style application takes 40 seconds to open its ports. Which probe protects it, and how?

3. Write the `resources` block that gives a container 100 millicores requested, 200 millicores limited and 256 Mi requested.
4. Which QoS class does a pod with a single container holding requests but no limits belong to?
5. A namespace has a ResourceQuota but no LimitRange. What happens to a pod that sets no requests?

Solutions

1. Readiness controls the endpoints; liveness restarts containers. Startup gates both.
2. A `startupProbe` with `failureThreshold: 20` and `periodSeconds: 2` allows 40 seconds while liveness stays dormant.
3. `yaml resources: requests: cpu: "100m" memory: "256Mi" limits: cpu: "200m"`
4. Burstable: requests are set but not equal to limits.
5. It is admitted without requests only if the quota has no request requirement; under a `requests.cpu` or `requests.memory` quota the pod is rejected with a `forbidden: exceeded quota` event.

7.6 Mini test

1. Which effect does a failing readiness probe have? A) the container is restarted B) the pod leaves the Service endpoints C) the pod is deleted D) the node is drained
2. What is the unit of `cpu: 250m`? A) 250 megabytes B) a quarter of a core C) 250 requests per second D) 250 threads
3. Which probe is designed for slow-booting applications? A) `readinessProbe` B) `livenessProbe` C) `startupProbe` D) `execProbe`
4. What happens when a container exceeds its memory limit? A) it is throttled B) it is OOM-killed C) it is throttled, then killed D) nothing
5. A pod has requests equal to limits on every container. Which QoS class is it? A) BestEffort B) Burstable C) Guaranteed D) Critical
6. True or false: exceeding a CPU limit restarts the container.
7. Which command shows current CPU and memory usage of pods? A) `kubectl describe pods` B) `kubectl top pods` C) `kubectl get pods` D) `kubectl logs pods`
8. Which object sets default requests inside a namespace? A) ResourceQuota B) LimitRange C) NetworkPolicy D) PriorityClass

Answer key and explanations

1. B - readiness gates traffic, it does not kill anything.
2. B - `m` means millicores.
3. C - startup probes tolerate long boot phases without guessing a delay.
4. B - memory cannot be throttled, so the runtime kills the process.
5. C - equal requests and limits on every container yields Guaranteed.

6. False - CPU is throttled, not restarted; only memory kills.
7. B - `top` reads the metrics API served by the metrics add-on.
8. B - LimitRange fills in missing requests and limits.

Chapter 8 - Namespaces, RBAC and pod security

8.1 Summary

- A namespace is the scope for names, quotas, network policy and role bindings.
- RBAC decides who may do what to which objects: role, subject, binding.
- A Role grants verbs inside one namespace; a ClusterRole grants them cluster-wide or on non-namespaced objects.
- RoleBinding and ClusterRoleBinding connect roles to users, groups and service accounts.
- Every pod runs as a service account, and that account is what its API callers see.
- The default service account is mounted into every pod unless automation is disabled.
- Pod Security Standards replace the older PodSecurityPolicy with three levels: privileged, baseline and restricted.
- Pod security levels are enforced by namespace labels, so policy follows the namespace, not the cluster.
- `securityContext` fields such as `runAsNonRoot`, `readOnlyRootFilesystem` and `seccompProfile` shrink the container's privileges.
- Least privilege means starting with restricted and loosening only where a workload proves it must.

8.2 Concepts

Namespaces as the unit of tenancy

output

```
one cluster
|
+-- namespace "prod"
|     deployments, services, quotas, roles
|
+-- namespace "stage"
|     same names allowed again, different objects
|
+-- namespace "kube-system"
      control-plane add-ons, only readable by default
```

The same manifest text deploys into every namespace; only the name of the namespace changes. That is the whole point.

RBAC: four verbs, three ideas

A permission is a verb on a resource. Roles carry the verbs; bindings attach them to subjects; service accounts carry the pod.

Subject	Meaning
User	a human, issued by the cluster's identity provider
Group	a set of users, matched by name
ServiceAccount	an identity for a process inside the cluster

YAML

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
rules:
  - apiGroups: [""]
    resources: ["pods", "pods/log"]
    verbs: ["get", "list", "watch"]
```

output

```
   Role "pod-reader"   (verbs x resources)
            |
            v
   RoleBinding           ---> subjects: default SA, deployer user
            |
            v
   effect: the pod may list and read pods in this namespace
```

ClusterRoles are used for cluster-scoped resources such as nodes, and for the built-in roles `view`, `edit`, `admin` and `cluster-admin`.

Pod Security Standards

Level	Promises	Typical namespace
`privileged`	no restrictions	system components
`baseline`	blocks known privilege escalations	shared, untrusted workloads
`restricted`	requires non-root, no privilege escalation, seccomp default	production workloads

Enforcement is a label on the namespace:

YAML

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: prod
  labels:
    pod-security.kubernetes.io/enforce: baseline
    pod-security.kubernetes.io/audit: restricted
    pod-security.kubernetes.io/warn: restricted
```

`enforce` rejects, `audit` records, `warn` tells the person applying the manifest. A third field, `enforce-version`, pins the policy version.

The security context

YAML

```yaml
securityContext:
  runAsNonRoot: true
  runAsUser: 1000
  seccompProfile:
    type: RuntimeDefault
containers:
  - name: web
    image: nginx:1.27
    securityContext:
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: ["ALL"]
```

Pod-level fields apply to every container; container-level fields refine one container. A read-only root filesystem forces writable paths onto volumes, which is the most common change when a workload moves to restricted.

8.3 Examples

Example 1: a namespace with an enforced level.

YAML

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: prod
  labels:
    pod-security.kubernetes.io/enforce: restricted
    pod-security.kubernetes.io/enforce-version: latest
```

shell

```shell
kubectl label namespace prod --overwrite pod-security.kubernetes.io/enforce=restricted
kubectl label --list namespace prod
```

output

```
namespace/prod labeled

pod-security.kubernetes.io/enforce=restricted
kubernetes.io/metadata.name=prod
```

Example 2: a pod rejected by the restricted level (not executed).

YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: root-pod
  namespace: prod
spec:
  containers:
    - name: web
      image: nginx:1.27
```

shell

```
kubectl apply -f root-pod.yaml
```

output

```
Error from server (Forbidden): error when creating "root-pod.yaml": pods "root-pod" is forbidden: viola
```

The message names every field the workload must set; that is how a restricted namespace teaches you.

Example 3: the same pod made compliant.

YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: safe-pod
  namespace: prod
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 10000
    seccompProfile:
      type: RuntimeDefault
  containers:
    - name: web
      image: nginx:1.27
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities:
          drop: ["ALL"]
      volumeMounts:
        - name: cache
          mountPath: /var/cache/nginx
        - name: run
          mountPath: /var/run
  volumes:
    - name: cache
      emptyDir: {}
    - name: run
      emptyDir: {}
```

Nginx writes pid and cache files at boot, so both paths become small `emptyDir` volumes and the image itself is untouched.

Example 4: a role and a binding for a service account.

YAML

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: deployer
  namespace: prod
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: deploy-writer
  namespace: prod
rules:
  - apiGroups: ["apps"]
    resources: ["deployments"]
    verbs: ["get", "list", "watch", "patch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: deployer-binding
  namespace: prod
subjects:
  - kind: ServiceAccount
    name: deployer
    namespace: prod
roleRef:
  kind: Role
  name: deploy-writer
  apiGroup: rbac.authorization.k8s.io
```

YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: ci-agent
  namespace: prod
spec:
  serviceAccountName: deployer
  containers:
    - name: agent
      image: myagent:3.0
```

Example 5: verify permissions without guessing (not executed).

shell

```shell
kubectl auth can-i update deployments -n prod --as=system:serviceaccount:prod:deployer
kubectl auth can-i list secrets -n prod --as=system:serviceaccount:prod:deployer
```

output

```
yes
no
```

`can-i --as` is the audit tool of RBAC: it answers exactly what the API server will answer.

8.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
`cluster-admin` for a CI account	one leak compromises everything	narrow Role, exact namespace
Using the default service account	every pod shares one identity	a service account per workload
Granting `list secrets` casually	credentials leak to app logs	grant secret reads only where needed
Labels that say `enforce: restricted` but a pod that needs root	deploys start failing at apply time	design restricted first, escalate per namespace
Pod security treated as optional hardening	a single root container becomes the incident	enforce baseline everywhere
Roles bound to users but baked in manifests	drift between Git and cluster	manage bindings as code and review them

- `kubectl config view --minify` and `kubectl auth whoami` tell you who you are before you debug a permission problem.
- Start every namespace with `audit: restricted` and `warn: restricted`; the warnings arrive while you write the manifest, not at 3 a.m.

8.5 Practice exercises

1. Write the labels that make namespace `prod` enforce `restricted` and audit `restricted`.
2. A CI service account may patch deployments but must not read secrets. Which objects do you create?
3. What does `kubectl auth can-i delete pods --as=system:serviceaccount:dev:builder` answer?
4. A restricted workload must write to `/tmp`. How do you keep the root filesystem read-only?
5. Why is a ClusterRoleBinding usually the wrong tool for an application?

Solutions

1. `yaml metadata: labels: pod-security.kubernetes.io/enforce: restricted pod-security.kubernetes.io/audit: restricted`
2. A Role (verbs `get,list,watch,patch` on `deployments`), a RoleBinding to the service account, and the service account itself; no secret verbs.
3. Whether that identity may delete pods, subject to its roles and bindings in the `dev` namespace: the API server answer, not an opinion.
4. Mount a small `emptyDir` at `/tmp` and set `readOnlyRootFilesystem: true`; alternatively use `--tmpfs` where the runtime supports it.
5. It grants cluster-wide reach to every namespace, so an application acting as that subject can interfere with any tenant, not just its own.

8.6 Mini test

1. Which object holds a set of permissions inside one namespace? A) RoleBinding B) Role C) ServiceAccount D) LimitRange
2. Which label enforces a Pod Security level? A) `pod-security.kubernetes.io/enforce` B) `security.kubernetes.io/level` C) `enforce/podsecurity` D) `k8s.io/security`
3. Which level forbids root execution? A) `privileged` B) `baseline` C) `restricted` D) both A and B
4. Which field of a pod names the service account it runs as? A) `serviceAccount` B) `serviceAccountName` C) `saRef` D) `automountServiceAccountToken`
5. True or false: `audit` rejects a manifest, while `enforce` only records it.
6. What does `kubectl auth can-i` tell you? A) whether the current user can run a command B) how many pods are running C) which role binds a namespace D) the cluster version
7. Which three Pod Security levels exist? A) low, medium, high B) privileged, baseline, restricted C) public, internal, private D) open, closed, sealed
8. What does `allowPrivilegeEscalation: false` prevent? A) writing to the root filesystem B) a process gaining more privileges than its parent C) mounting volumes D) binding port 80

Answer key and explanations

1. B - a Role lists verbs and resources; the binding attaches subjects.
2. A - the three label fields are `enforce`, `audit` and `warn`.
3. C - restricted requires non-root, no escalation and a default seccomp profile.
4. B - the pod then authenticates as that service account.
5. False - `enforce` rejects, `audit` records in the audit log.
6. A - it simulates an authorisation decision for a user or service account.
7. B - the Pod Security Standards are privileged, baseline and restricted.
8. B - it blocks setuid-style escalation inside the container.

Chapter 9 - Ingress and external access

9.1 Summary

- An Ingress is a set of routing rules for HTTP and HTTPS traffic entering the cluster.
- An Ingress needs a controller: the object alone is only configuration for that controller.
- Ingress resources match by host and path, then forward to a Service by name and port.
- `ingressClassName` selects the controller that must implement the rules.
- TLS is configured per host with a Secret holding the certificate and key.
- A `LoadBalancer` Service is the other public entry point, one address per service.
- An Ingress gives one address and host-based routing for many services.
- `ExternalIPs` on a Service is a manual way to publish a Service address.
- Rewrites, sticky sessions and rate limits live in controller-specific annotations.

- Ingress is layer 7; Services are layer 4, which is why Ingress can route by path and terminate TLS.

9.2 Concepts

Where Ingress sits

output

```
   internet
      |
      v
+------------------+      +-----------+        +------------+        +---------+
| load balancer /  | ---> | ingress   | ---->  | Service    | ---->  | pods    |
| node port on the |      | controller|        | ClusterIP  |        |         |
| controller pods  |      +-----------+        +------------+        +---------+
+------------------+
      rules: host shop.example.com
             path /api -> api-svc:80
             path /    -> web-svc:80
             TLS from secret shop-tls
```

The controller watches Ingress objects, programs its proxy (nginx, Envoy, a cloud load balancer) and keeps the routing table in sync as Services change.

Ingress versus Services

Need	Use
one public address, many hosts and paths	Ingress
a single public port for one service, TCP or UDP	Service type `LoadBalancer`
internal cluster traffic only	Service type `ClusterIP`
node-local experiments	Service type `NodePort`
a DNS name for an external dependency	Service type `ExternalName`

Anatomy of an Ingress

Field	Meaning
`ingressClassName`	the controller that must implement this object
`rules[].host`	the host name this rule answers for
`rules[].http.paths[]`	path patterns with a `backend`
`paths[].pathType`	`Prefix`, `Exact` or `ImplementationSpecific`
`paths[].backend.service`	Service name and port
`tls[]`	host names and the Secret holding the certificate

`pathType: Prefix` matches `/api` and everything below it; `Exact` matches only the literal path.

TLS at the edge

The certificate lives in a `kubernetes.io/tls` Secret, and the Ingress references it by host:

YAML

```yaml
tls:
  - hosts:
      - shop.example.com
    secretName: shop-tls
```

Certificates are terminated at the controller, so the traffic between the controller and the pod usually stays inside the cluster network.

9.3 Examples

Example 1: route two services on one host.

YAML

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
    - host: shop.example.com
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: api
                port:
                  number: 80
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web
                port:
                  number: 80
```

With minikube, enable the controller once: `minikube addons enable ingress`.

Example 2: publish and test (not executed).

shell

```shell
kubectl apply -f ingress.yaml
kubectl get ingress shop
```

output

```
NAME    CLASS   HOSTS              ADDRESS        PORTS     AGE
shop    nginx   shop.example.com   192.168.49.2   80, 443   12s
```

shell

```
curl -H "Host: shop.example.com" http://192.168.49.2/api/health
```

output

```
{"status":"ok"}
```

The `Host` header is what the controller matches on, so a direct IP call needs it while a real name works through DNS.

Example 3: terminate TLS at the Ingress.

YAML

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: shop-tls
type: kubernetes.io/tls
data:
  tls.crt: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCg==
  tls.key: LS0tLS1CRUdJTiBSU0EgUFJJVkFURSBLRVktLS0tLQo=
```

YAML

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: shop-tls
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - shop.example.com
      secretName: shop-tls
  rules:
    - host: shop.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: web
                port:
                  number: 80
```

Both fragments can live in one file separated by `---`; the Ingress resource is the object the controller reads.

Example 4: a LoadBalancer Service instead (not executed).

YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-lb
spec:
  type: LoadBalancer
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 8080
```

shell

```
kubectl get svc web-lb -w
```

output

```
NAME      TYPE           CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
web-lb    LoadBalancer   10.96.44.10    <pending>     80:31234/TCP   4s
web-lb    LoadBalancer   10.96.44.10    203.0.113.7   80:31234/TCP   31s
```

On a bare-metal or local cluster `EXTERNAL-IP` stays `pending` unless a load balancer implementation is installed, because there is no cloud API to call.

Example 5: read the condition of an Ingress (not executed).

shell

```
kubectl describe ingress shop
```

output

```
Name:             shop
Labels:           <none>
Namespace:        default
Address:          192.168.49.2
Ingress Class:    nginx
TLS:
  shop-tls terminates shop.example.com
Rules:
  Host               Path  Backends
  ----               ----  --------
  shop.example.com
                     /api     api:80 (10.244.0.9:8080,10.244.1.3:8080)
                     /        web:80 (10.244.0.12:8080,10.244.2.7:8080)
Events:
  Type    Reason  Age   From                      Message
  ----    ------  ----  ----                      -------
  Normal  Sync    15s   nginx-ingress-controller  Scheduled for sync
```

The event log shows the controller's view; an empty backend list means the Service selector matched nothing.

9.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
No ingress controller installed	the object exists but nothing routes	install one, or use `LoadBalancer` services
IngressClassName missing or wrong	the object is ignored or handled by the wrong controller	set it explicitly
Overlapping paths	routing follows controller-specific precedence	keep paths disjoint or document the order
Certificate in the repo	the TLS key leaks with the code	inject the Secret from a secret store
Assuming Ingress replaces Services	clients inside the cluster bypass it	keep both, ingress faces out, ClusterIP faces in
Trailing slash expected in health checks	the probe path does not match the rule	use `pathType: Exact` where the path is fixed

- Put the DNS name in the manifest, not the IP: IPs of controllers change.
- Controllers differ in annotations; read the documentation of the class you actually run.

9.5 Practice exercises

1. An Ingress object is applied but requests time out. Give the first three things you check.
2. Write an Ingress rule that sends `shop.example.com/static` to the Service `assets` on port 8080 with `pathType: Prefix`.
3. What does `ingressClassName: nginx` do?
4. Which Secret type does an Ingress expect for TLS, and which keys must it hold?
5. Why is an Ingress better than publishing ten `NodePort` services?

Solutions

1. That an ingress controller is installed and its pods are running; that `ingressClassName` matches that controller; that the backend Service has endpoints and the port matches (`kubectl describe ingress` shows the backends).
2. `yaml - path: /static pathType: Prefix backend: service: name: assets port: number: 8080`
3. It selects which controller must implement the rules of this object; without it the object may be ignored.
4. `kubernetes.io/tls`, holding `tls.crt` and `tls.key`.
5. One address with host and path routing, TLS termination and a single place to audit traffic, instead of ten random high ports with no shared entry point.

9.6 Mini test

1. Which object routes HTTP traffic by host and path? A) Service B) Ingress C) Endpoints D) NetworkPolicy

2. What must exist for an Ingress to work? A) a certificate B) an ingress controller C) a NodePort service D) a PV
3. Which `pathType` matches only the literal path? A) `Prefix` B) `Exact` C) `ImplementationSpecific` D) `Literal`
4. Where is TLS traffic terminated in a typical Ingress setup? A) at the pod B) at the ingress controller C) at etcd D) at the kubelet
5. True or false: an Ingress replaces the Service in front of the pods.
6. Which field selects the controller for an Ingress object? A) `spec.class` B) `spec.ingressClassName` C) `metadata.labels.controller` D) `spec.type`
7. On a local cluster with no cloud, what does `EXTERNAL-IP` of a `LoadBalancer` Service usually show? A) `127.0.0.1` B) `<pending>` C) the node IP D) an error
8. Which two resources does an Ingress rule need in its backend? A) a pod name and node port B) a Service name and port C) an Endpoints name and port D) a Deployment name and port

Answer key and explanations

1. B - Ingress is the layer-7 routing object.
2. B - the object is configuration; the controller does the routing.
3. B - `Exact` matches the whole path and nothing else.
4. B - the controller decrypts and forwards plainly inside the cluster unless re-encryption is configured.
5. False - the Ingress forwards to a Service, which still holds the endpoints.
6. B - `ingressClassName` names the implementing controller.
7. B - nothing provisions an address without a cloud or a local load balancer implementation.
8. B - backends reference a Service by name and port.

Chapter 10 - Helm and packaging

10.1 Summary

- Helm packages Kubernetes manifests into a versioned, reusable unit called a chart.
- A chart is a directory with `Chart.yaml`, a `values.yaml` file and templates rendered with Go templates.
- Installing a chart creates a release; a revision is one snapshot of that release.
- Values override the template defaults, and each override path ends in the rendered manifest.
- `helm template` renders a chart locally without a cluster, which is ideal for review and diffing.
- Repositories are HTTP indexes of charts; `helm repo add` registers one.
- `helm upgrade --install` is the idempotent way to apply a release.
- `helm rollback` returns to a previous revision, using the stored release history.

- Chart dependencies live in `Chart.yaml` and are vendored with `helm dependency update`.
- A chart's `Chart.yaml` carries `apiVersion: v2`, a `version` and an `appVersion`.

10.2 Concepts

What a chart contains

output

```
mychart/
|-- Chart.yaml          name, version, appVersion, dependencies
|-- values.yaml         defaults for every template
|-- charts/             vendored dependencies
|-- templates/
|   |-- _helpers.tpl    named template parts
|   |-- deployment.yaml
|   |-- service.yaml
|   |-- ingress.yaml
|   |-- NOTES.txt       printed after install
`-- .helmignore
```

`Chart.yaml` is the identity of the chart:

YAML

```yaml
apiVersion: v2
name: mychart
description: A demo chart for the study volume
type: application
version: 0.2.0
appVersion: "1.27"
```

The chart `version` is the package version; `appVersion` names the software the chart installs.

Values and the template context

A template reads values through the `.Values` object, with chart metadata in `.Chart` and the release in `.Release`:

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-web
  labels:
    app: {{ .Chart.Name }}
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
        - name: web
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
```

The usual values file for that template:

YAML

```yaml
replicaCount: 2
image:
  repository: nginx
  tag: "1.27"
```

`{{- ... -}}` trims the surrounding whitespace, which is what keeps the rendered YAML aligned.

The release model

Command	Effect
`helm install web ./mychart`	create release `web`, revision 1
`helm upgrade web ./mychart -f prod.yaml`	revision 2 with new values
`helm history web`	list revisions with status
`helm rollback web 1`	re-apply revision 1
`helm uninstall web`	delete the release, keep the history

`helm upgrade --install` installs when the release is missing and upgrades when it exists, which makes pipeline scripts simple.

10.3 Examples

Example 1: a minimal chart, two files and a template.

YAML

```yaml
apiVersion: v2
name: webchart
description: A tiny web chart
version: 0.1.0
appVersion: "1.27"
```

YAML

```yaml
replicaCount: 2
image:
  repository: nginx
  tag: "1.27"
service:
  port: 80
```

YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-web
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: {{ .Release.Name }}-web
  template:
    metadata:
      labels:
        app: {{ .Release.Name }}-web
    spec:
      containers:
        - name: web
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-web
spec:
  selector:
    app: {{ .Release.Name }}-web
  ports:
    - port: {{ .Values.service.port }}
      targetPort: 80
```

Example 2: render without installing (not executed).

shell

```
helm template web ./webchart -f prod.yaml
```

`helm template` prefixes every manifest with a `# Source:` comment naming the file it came from; those comment lines are trimmed here.

output

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-web
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web-web
  template:
    metadata:
      labels:
        app: web-web
    spec:
      containers:
        - name: web
          image: "nginx:1.27"
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: web-web
spec:
  selector:
    app: web-web
  ports:
    - port: 80
      targetPort: 80
```

`helm template` output is plain Kubernetes YAML: review it, then apply with `kubectl apply -f` if a pipeline prefers that.

Example 3: install, list and inspect (not executed).

shell

```
helm install web ./webchart
helm list
helm get manifest web | head -12
```

output

```
NAME: web
LAST DEPLOYED: Sun Oct 10 09:14:22 2026
NAMESPACE: default
STATUS: deployed
REVISION: 1
TEST SUITE: None

NAME□NAMESPACE□REVISION□UPDATED                                  □STATUS  □CHART           □APP VERSION
web  □default  □1       □2026-10-10 09:14:22.411302 +0200 CEST   □deployed□webchart-0.1.0 □1.27
```

`helm get manifest` prints exactly what was installed, which is the first thing to check when a release misbehaves.

Example 4: upgrade and roll back (not executed).

shell

```
helm upgrade web ./webchart --set replicaCount=4
helm history web
helm rollback web 1
```

output

```
Release "web" has been upgraded. Helm Revision: 2

REVISION□UPDATED                  □STATUS    □CHART           □DESCRIPTION
1       □Sun Oct 10 09:14:22 2026□superseded□webchart-0.1.0□Install complete
2       □Sun Oct 10 09:20:41 2026□deployed   □webchart-0.1.0□Upgrade complete

Rollback was a success! Helm bohemian dog jukebox is now deployed again.
```

(The trailing sentence is Helm's playful release-note message; upgrade and rollback themselves are the parts to remember.)

Example 5: install a published chart from a repository (not executed).

shell

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
helm search repo bitnami/nginx --versions | head -3
```

output

```
"bitnami" has been added to your repositories
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "bitnami" repository
Update Complete. âŽˆHappy Helming!âŽˆ

bitnami/nginx        22.1.2         20.0.4         NGINX Open Source is a web server...
```

`helm show values bitnami/nginx` prints every value a chart accepts; that is the documentation of the chart's interface.

10.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Editing manifests inside `templates` ad hoc	every release diverges	express everything as values
One values file per environment in the chart	merge order surprises	use a base file plus `-f env.yaml`
`helm upgrade` with no history	no way back	keep the default history, roll back by revision
Rendering without `--atomic` on a critical service	a failed upgrade leaves a broken release	use `--atomic --timeout 5m`
Versioning only `appVersion`	environments drift silently	bump the chart `version` on every change
Secrets committed in `values.yaml`	the chart leaks credentials	reference a Secret, or use a secret-aware tool

- `helm lint ./chart` catches template errors before a pipeline does.
- `helm diff upgrade` (a plugin) prints the change before you apply it.

10.5 Practice exercises

1. Name the four elements a chart directory must contain to install.
2. Which command renders a chart to standard output without touching the cluster?
3. Write the template expression that renders `replicas` from a value named `replicaCount`.
4. What is the difference between `version` and `appVersion` in `Chart.yaml`?
5. Which two flags make an upgrade roll itself back on failure, and why would a pipeline want them?

Solutions

1. `Chart.yaml` (the metadata), `values.yaml` (defaults), a `templates/` directory with at least one manifest, and a valid chart `version`.
2. `helm template <release> ./chart`.
3. `replicas: {{ .Values.replicaCount }}`.
4. `version` is the version of the chart package itself and must rise on every chart change; `appVersion` names the software being installed and changes only when that software changes.
5. `--atomic` and a `--timeout`: atomic rolls the release back if the upgrade fails or the timeout expires, which is what a pipeline wants when a broken revision would otherwise stay.

10.6 Mini test

1. Which file holds the chart's identity? A) `values.yaml` B) `Chart.yaml` C) `templates.yaml` D) `Chartfile`
2. Which command creates a release? A) `helm create` B) `helm install` C) `helm package` D) `helm repo`
3. Which template object carries user values? A) `.Chart` B) `.Release` C) `.Values` D) `.Template`
4. What does `helm rollback web 1` do? A) deletes the release B) re-applies revision 1 of the release C) downgrades Helm itself D) renames the release
5. True or false: `helm template` needs a working cluster connection.
6. Which file is printed to the user after a successful install when it exists? A) `NOTES.txt` B) `README.md` C) `values.yaml` D) `Chart.yaml`
7. What does `helm repo add` do? A) installs a chart B) registers a chart repository C) creates a chart D) packages a chart
8. Why must the chart `version` change on every chart edit? A) Kubernetes requires it B) so releases are identifiable and upgrades are predictable C) to make the image smaller D) it does not need to

Answer key and explanations

1. B - `Chart.yaml` names and versions the package.
2. B - `helm install` deploys a chart as a release.
3. C - `.Values` is the merged values file.
4. B - rollback restores a stored revision of the same release.
5. False - it renders locally and prints manifests, which is why it is safe in review pipelines.
6. A - `NOTES.txt` shows post-install hints such as how to reach the app.
7. B - repositories are indexes of charts, added by name.
8. B - revisions must be distinguishable for upgrade, rollback and audit.

Chapter 11 - Observability basics

11.1 Summary

- Kubernetes gives you logs, events and metrics before any monitoring stack exists.
- `kubectl logs` prints a container's stdout and stderr; `-f` follows, `--previous` reads a dead container.
- `kubectl get events` lists what the cluster did, sorted, which is where failures are explained.
- `kubectl describe` compiles an object's spec, status and events into one report.
- A container that logs to files needs a sidecar that streams them to stdout.
- The logging sidecar pattern keeps one log destination and one format per application.
- The metrics API feeds `kubectl top`; it needs the metrics-server add-on on a local cluster.
- Structured JSON logs make automated parsing and structured queries possible.
- Health of a pod is not health of the application: probes plus logs plus events tell the whole story.
- For long-term storage, log and metric data leave the cluster to a dedicated system.

11.2 Concepts

The default troubleshooting path

output

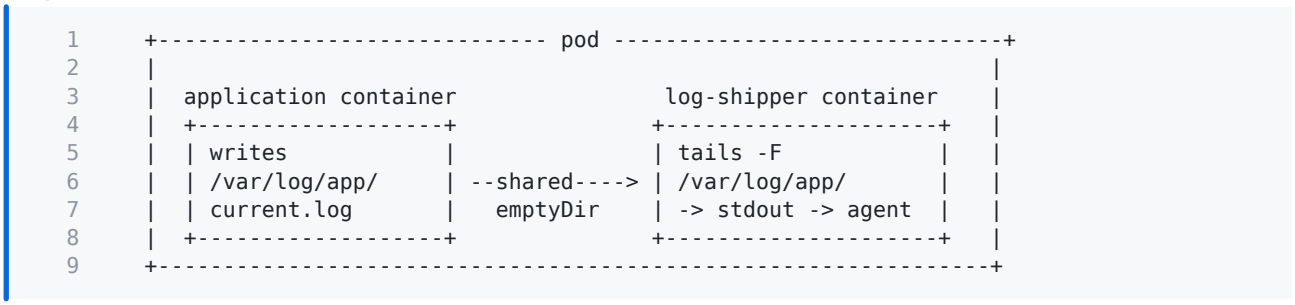
```
something is wrong
       |
       v
kubectl get pods            which pod, what status?
       |
       v
kubectl describe pod        what did the cluster try, what failed?
       |
       v
kubectl logs <pod> [-c c]   what did the application say?
       |
       v
kubectl get events          what happened around that time?
       |
       v
kubectl top / metrics       is it a resource problem?
```

Events expire from the API after a while, so capture them early in an incident.

The sidecar logging pattern

An application writing to files inside the container leaves no log stream to collect. A second container in the same pod tails the file and writes to stdout, where the node's log agent can pick it up.

output



The two containers share one `emptyDir` volume; the sidecar never modifies the application image.

What is built in and what is not

Signal	Built in	Needs more
application logs	`kubectl logs`, stdout and stderr per container	aggregation and retention
cluster events	`kubectl get events`	an events exporter for history
node and pod metrics	metrics API through metrics-server	Prometheus or a cloud service
traces	none	an OpenTelemetry collector and instrumented apps
dashboards and alerts	none	an observability stack

11.3 Examples

Example 1: the sidecar in a manifest.

YAML

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: logged-app
spec:
  volumes:
    - name: logs
      emptyDir: {}
  containers:
    - name: app
      image: myapp:1.2
      command: ["sh", "-c", "while true; do date >> /var/log/app/current.log; sleep 5; done"]
      volumeMounts:
        - name: logs
          mountPath: /var/log/app
    - name: shipper
      image: busybox:1.36
      command: ["sh", "-c", "tail -n+1 -F /var/log/app/current.log"]
      volumeMounts:
        - name: logs
          mountPath: /var/log/app
```

The sidecar starts with the pod and keeps the log stream available on stdout.

Example 2: read both log streams (not executed).

shell

```shell
kubectl logs logged-app -c app --tail=2
kubectl logs logged-app -c shipper --tail=2
```

output

```
Sun Oct 10 09:31:04 UTC 2026
Sun Oct 10 09:31:09 UTC 2026

Sun Oct 10 09:31:04 UTC 2026
Sun Oct 10 09:31:09 UTC 2026
```

Same lines, different sources: the sidecar mirrors the file to the standard stream every log agent reads.

Example 3: the event stream (not executed).

shell

```shell
kubectl get events --sort-by=.lastTimestamp | tail -6
```

output

```
LAST SEEN   TYPE      REASON      OBJECT              MESSAGE
61s         Normal    Scheduled   pod/logged-app      Successfully assigned default/logged-app to minik
60s         Normal    Pulled      pod/logged-app      Container image "myapp:1.2" already present on ma
59s         Normal    Created     pod/logged-app      Created container app
59s         Normal    Started     pod/logged-app      Started container app
40s         Warning   Unhealthy   pod/logged-app      Readiness probe failed: Get "http://10.244.0.9:80
58s         Normal    Started     pod/logged-app      Started container shipper
```

Events are the cluster's diary: scheduling, pulls, probes and restarts all appear with timestamps.

Example 4: one report per object (not executed).

shell

```
kubectl describe pod logged-app | sed -n '1,25p'
```

output

```
Name:             logged-app
Namespace:        default
Priority:         0
Service Account:  default
Node:             minikube/192.168.49.2
Start Time:       Sun, 10 Oct 2026 09:30:03 +0000
Labels:           <none>
Annotations:      <none>
Status:           Running
IP:               10.244.0.9
Containers:
  app:
    Container ID:  containerd://7c1f9a2b4d5e
    Image:         myapp:1.2
    Image ID:      docker.io/library/myapp@sha256:9f2...
    Port:          <none>
    Host Port:     <none>
    State:          Running
      Started:      Sun, 10 Oct 2026 09:30:04 +0000
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/log/app from logs (rw)
```

Everything a support ticket needs is in one screen: node, image, mount, ready state and restart count.

Example 5: metrics and memory pressure (not executed).

shell

```
kubectl top pod logged-app
kubectl top nodes
```

output

```
NAME         CPU(cores)   MEMORY(bytes)
logged-app   3m           18Mi

NAME       CPU(cores)   CPU%   MEMORY(bytes)   MEMORY%
minikube   198m         9%     1390Mi          36%
```

A pod whose memory climbs toward its limit is a future OOM kill; `kubectl top` is the cheapest way to see it coming.

11.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Logging to files inside the container	nothing can collect it	write to stdout and stderr
Two loggers per pod with different formats	searching is guesswork	one format, one destination
Deciding on metrics alone	an app can be busy and broken	correlate logs, events and probes
Debugging with `kubectl exec` only	you never see what the cluster saw	read `describe` and events first
Unbounded `print` statements	I/O throttling and huge bills	log at levels, sample the noisy parts
Ignoring `--previous`	the crash logs are gone after restart	always read the previous container first

- Log requests with a correlation id so a single user action is traceable across services.
- Keep one JSON line per event; line-oriented, machine-parseable logs are what every aggregator expects.

11.5 Practice exercises

1. An application writes only to `/var/log/app/current.log`. Describe a pod layout that makes those lines available to `kubectl logs`.
2. Which command shows the last events in the cluster, newest first?
3. A pod restarts twice overnight. Which command shows the logs of the run that crashed?
4. What does `kubectl top pods` need in order to work?
5. Why are the timestamps in `kubectl get events` worth copying before anything else?

Solutions

1. Add a second container to the same pod that mounts the same volume and runs something like `tail -F /var/log/app/current.log`, writing to stdout.
2. `kubectl get events --sort-by=.lastTimestamp | tail -n 20`.
3. `kubectl logs <pod> -c <container> --previous`.
4. A metrics provider: the metrics API, usually served by metrics-server, must be installed in the cluster.
5. Events have a retention window in the API, so once they age out the explanation of what happened disappears with them.

11.6 Mini test

1. Which stream does `kubectl logs` print? A) files under `/var/log` B) the container's stdout and stderr C) the node's journal D) etcd entries
2. Which flag shows the logs of a container that has already restarted? A) `-c` B) `--tail` C) `--previous` D) `--follow`
3. How do you read the logs of one container in a multi-container pod? A) `kubectl logs pod/container` B) `kubectl logs pod -c container` C) `kubectl logs pod --all` D) `kubectl exec pod logs`
4. Which command sorts events by recency? A) `kubectl get events --sort-by=.lastTimestamp` B) `kubectl logs -f` C) `kubectl describe nodes` D) `kubectl top events`
5. True or false: a sidecar can share a volume with the application container to stream its logs.
6. What does the sidecar in the logging pattern write to? A) a database B) the container's stdout C) the node's disk directly D) a PersistentVolume
7. Which two signals does `kubectl top` report? A) requests and limits B) CPU and memory usage C) latency and errors D) restarts and events
8. Why do structured JSON logs help? A) they are smaller B) tools can query fields without parsing text C) Kubernetes requires them D) they disable buffering

Answer key and explanations

1. B - Kubernetes captures the container's standard streams.
2. C - `--previous` reads the terminated container's logs.
3. B - `-c` names the container.
4. A - the sort field `.lastTimestamp` orders by recency.
5. True - a shared volume is the hand-off channel.
6. B - the sidecar converts a file into a log stream.
7. B - `top` reports current CPU and memory from the metrics API.
8. B - fields are queryable, so dashboards and alerts do not depend on regexes.

Chapter 12 - Rolling updates, rollbacks and release strategies

12.1 Summary

- A rolling update replaces pods gradually so the application keeps serving traffic.
- `maxSurge` and `maxUnavailable` decide how fast and how safe a rollout is.
- `kubectl rollout status`, `history`, `pause`, `resume` and `undo` control and inspect a rollout.
- A failed rollout that does not converge within `progressDeadlineSeconds` is marked failed automatically.

- Canary deployments send a small share of traffic to a new version before the switch.
- Blue/green keeps two full environments and flips traffic once at a time.
- Readiness probes are what make any of this safe: traffic only reaches ready pods.
- A rollback creates a new revision from an older template, not a literal undo.
- Immutable images and a clean template history make rollbacks meaningful.
- Pausing a rollout lets you verify one batch before the controller continues.

12.2 Concepts

The rollout controller

A Deployment keeps one ReplicaSet per template revision and scales them in opposite directions:

output

```
update starts
     |
     v
new RS scaled up ------------------------>  new RS: 3
old RS scaled down                          old RS: 2
     |                                            |
readiness gates traffic                           v
     |                                      new RS: 3, old RS: 0
     v
rollout complete (new RS holds all replicas)
```

With `maxSurge: 1` and `maxUnavailable: 0` the sequence for three replicas is: create one new pod, wait for it to be ready, terminate one old pod, repeat.

The strategy numbers

Setting	Meaning	Effect
`maxSurge: 25%`	extra pods allowed above the desired count	faster rollout, more capacity briefly
`maxUnavailable: 25%`	pods that may be down	faster rollout, less capacity briefly
`maxSurge: 0`, `maxUnavailable: 1`	in-place only	slowest and safest for capacity

Both fields are percentages or integers, and the effective values round up for surge and down for unavailability.

Release strategies compared

Strategy	Traffic shape	Risk	Cost
Rolling	gradual shift inside one Deployment	contained by probes	normal

Strategy	Traffic shape	Risk	Cost
Canary	small share to the new version	contained by the share	two deployments plus routing
Blue/green	all traffic flips at once	blast radius at the flip	double capacity
Recreate	stop, then start	downtime	lowest

Canary on plain Kubernetes means two Deployments behind one Service, or an Ingress that splits traffic by weight. The Service selector is the switch: point it at the new labels and traffic follows immediately.

12.3 Examples

Example 1: trigger and watch a rolling update (not executed).

shell

```
kubectl set image deployment/web web=nginx:1.28
kubectl rollout status deployment/web
```

output

```
deployment.apps/web image updated

Waiting for deployment "web" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "web" rollout to finish: 1 of 3 updated replicas are available...
Waiting for deployment "web" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "web" rollout to finish: 2 of 3 updated replicas are available...
deployment "web" successfully rolled out
```

The status line counts pods, not time; a rollout that stops progressing is a rollout to investigate.

Example 2: history, pause and resume (not executed).

shell

```
kubectl rollout history deployment/web
kubectl rollout pause deployment/web
kubectl rollout resume deployment/web
```

output

```
deployment.apps/web
REVISION  CHANGE-CAUSE
1         nginx 1.27
2         nginx 1.28

deployment "web" paused

Waiting for deployment "web" rollout to finish: 2 out of 3 new replicas have been updated...
deployment "web" successfully rolled out
```

Pausing leaves the ReplicaSets at their current scale, so a batch sits in place until you resume.

Example 3: roll back one revision (not executed).

shell

```
kubectl rollout undo deployment/web
kubectl get rs -l app=web
```

output

```
deployment.apps/web rolled back

NAME             DESIRED   CURRENT   READY   AGE
web-6b4f8d99c7   3         3         3       12m
web-7d9f8c4b     0         0         0       15m
```

Rollback re-scales the old ReplicaSet to the desired count; the new one goes to zero. Both stay for history.

Example 4: a canary pair behind one Service.

YAML

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-stable
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
      track: stable
  template:
    metadata:
      labels:
        app: web
        track: stable
    spec:
      containers:
        - name: web
          image: nginx:1.27
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-canary
spec:
  replicas: 1
  selector:
    matchLabels:
      app: web
      track: canary
  template:
    metadata:
      labels:
        app: web
        track: canary
    spec:
      containers:
        - name: web
          image: nginx:1.28
```

YAML

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web
spec:
  selector:
    app: web
  ports:
    - port: 80
      targetPort: 80
```

Both Deployments carry `app: web`, so the Service load-balances across four pods and roughly a quarter of requests hit the canary. Scaling `web-canary` up and `web-stable` down promotes the new version gradually.

Example 5: the same idea, promoted with a selector flip.

shell

```
kubectl patch service web --type=json -p '[{"op":"add","path":"/spec/selector/track","value":"canary"}]
```

output

```
service/web patched
```

The flip is instant and complete, which is why blue/green needs a second reversible step before the old environment is scaled away.

12.4 Common mistakes and best practices

Mistake	Why it hurts	Best practice
Readiness probe that always passes	traffic reaches pods that cannot serve	probe a real dependency-free endpoint
`maxUnavailable: 100%`	full outage during every deploy	keep unavailability at zero or a small percentage
No `progressDeadlineSeconds` review	a stuck rollout looks healthy	alert on failed rollouts
Rolling back by hand-editing the template	the history is lost	`kubectl rollout undo`
Mutable image tags	the same tag changes meaning	push immutable digests or unique tags
Canary without a metric to watch	you cannot tell if it is worse	watch error rate and latency on the canary pods
Forgetting `strategy` for stateful apps	in-place replacement can corrupt state	use StatefulSets and delete pods one at a time

- Every deploy is a rollback waiting to happen: keep the chart or manifest that produced the previous release.
- Record the cause of each revision so the history is readable later.

12.5 Practice exercises

1. A Deployment with three replicas uses `maxSurge: 1`, `maxUnavailable: 0`. Describe the pod count during the rollout.
2. Which command returns a Deployment to its previous template, and what does it create?
3. A canary is 1 of 10 pods. Which two commands promote it without moving the Service selector?
4. Why does a broken readiness probe turn a rolling update into a stalled one?
5. What does `progressDeadlineSeconds: 600` guarantee?

Solutions

1. At most four pods exist (three plus one surge) and never fewer than three: one new pod must become ready before an old one is removed.
2. `kubectl rollout undo deployment/web`: the controller creates a new revision whose template comes from the previous one and scales the ReplicaSets accordingly.
3. `kubectl scale deployment/web-stable --replicas=8` followed by `kubectl scale deployment/web-canary --replicas=2`, repeated until the canary holds the traffic.
4. New pods never become ready, so the Deployment cannot terminate old pods and the rollout stays incomplete until the deadline marks it failed.
5. Nothing beyond a report: after ten minutes without progress the Deployment is marked `ProgressDeadlineExceeded`, which is the signal to alert on.

12.6 Mini test

1. Which field caps how many extra pods a rollout may create? A) `maxUnavailable` B) `maxSurge` C) `revisionHistoryLimit` D) `replicas`
2. Which probe must pass before a new pod receives Service traffic in a rolling update? A) liveness B) readiness C) startup D) none
3. What does `kubectl rollout undo deployment/web` do? A) deletes the Deployment B) applies a new revision based on an older one C) restarts every pod with the same image D) reverts the ConfigMap
4. Which release strategy keeps two full environments and flips traffic at once? A) rolling B) canary C) blue/green D) recreate
5. True or false: a canary on plain Kubernetes is usually two Deployments behind one Service.
6. Which pair of commands promotes a canary that shares a Service selector? A) `scale` the stable up and the canary down B) `scale` the canary up and the stable down C) delete the Service and recreate it D) patch the Service `clusterIP`
7. What happens when a rollout makes no progress for `progressDeadlineSeconds`? A) the cluster rolls back automatically B) the Deployment is marked as failed, but nothing is reverted C) the node is drained D) the pods are frozen
8. Which strategy causes downtime by design? A) RollingUpdate B) `Recreate` C) canary D) blue/green

Answer key and explanations

1. B - surge is the headroom above the desired count.
2. B - readiness gates the endpoints, so a new pod waits until it is ready.
3. B - the rollback is itself a new revision built from an older template.
4. C - blue/green holds both environments and switches the selector.
5. True - shared labels give the Service one pool to balance over.
6. B - scale the new version up while the old one drains.
7. B - the deadline is a signal, not an action; deciding what to do next is yours.
8. B - the `Recreate` strategy deletes old pods before creating new ones.

Cheat sheet

13.1 Cluster and objects

Task	Command
create a local cluster	`minikube start` or `kind create cluster`
cluster info	`kubectl cluster-info`
nodes	`kubectl get nodes -o wide`
contexts	`kubectl config get-contexts`
apply a manifest	`kubectl apply -f file.yaml`
delete by manifest	`kubectl delete -f file.yaml`
dry-run to YAML	`kubectl create deploy x --image=nginx --dry-run=client -o yaml`
field documentation	`kubectl explain pod.spec.containers`
watch changes	`kubectl get pods -w`

13.2 Object anatomy

Field	Meaning
`apiVersion`	group and version of the API
`kind`	the object type
`metadata.name`	unique name inside the namespace
`metadata.labels`	identifying key-value pairs
`metadata.annotations`	non-identifying metadata

Field	Meaning
`spec`	the desired state you write
`status`	the observed state the system writes
`ownerReferences`	the controller that will delete this object

13.3 Pods and workloads

Object	Use it for	Key fields
Pod	learning, debugging, jobs	`containers`, `restartPolicy`
ReplicaSet	fixed pod count (rarely written directly)	`replicas`, `selector`, `template`
Deployment	stateless applications	`strategy`, `revisionHistoryLimit`
StatefulSet	stable names and storage	`serviceName`, `volumeClaimTemplates`
Job / CronJob	one-shot and scheduled work	`completions`, `schedule`

13.4 Services

Type	Reaches	Remember
`ClusterIP`	inside the cluster	default
`NodePort`	`nodeIP:30000-32767`	also has a ClusterIP
`LoadBalancer`	provisioning an external address	needs a cloud or metal LB
`ExternalName`	a DNS alias	no endpoints, no proxy
headless	pod addresses via DNS	`clusterIP: None`

DNS: `service`, `service.namespace`, `service.namespace.svc.cluster.local`.

13.5 Configuration and storage

Need	Object
plain configuration	ConfigMap
env from all keys	`envFrom.configMapRef`
env from one key	`valueFrom.configMapKeyRef`
files inside the container	a volume from `configMap` or `secret`
credentials	Secret with `stringData` or `data`
ephemeral shared space	`emptyDir`
durable storage	PVC bound to a PV

Need	Object
per-replica durable storage	`volumeClaimTemplates` in a StatefulSet

13.6 Probes and resources

Probe	On failure
`startupProbe`	container restarts, others wait
`readinessProbe`	pod leaves the Service endpoints
`livenessProbe`	container is killed and restarted

Field	Enforced by
`requests.cpu` / `requests.memory`	scheduling and eviction ranking
`limits.cpu`	CPU throttling
`limits.memory`	OOM kill

QoS: Guaranteed (requests equal limits), Burstable (some requests), BestEffort (none).

13.7 Security quick reference

Thing	Object or field
namespace scoping	`metadata.namespace`
permission set in a namespace	Role
permission set cluster-wide	ClusterRole
attach permissions	RoleBinding / ClusterRoleBinding
pod identity	ServiceAccount in `spec.serviceAccountName`
check a decision	`kubectl auth can-i <verb> <resource> --as=<subject>`
pod policy levels	`pod-security.kubernetes.io/enforce: privileged\ baseline \ restricted `
drop privileges	`runAsNonRoot`, `allowPrivilegeEscalation: false`, `capabilities.drop: [ALL]`, `seccompProfile.type: RuntimeDefault`

13.8 Helm

Task	Command
scaffold a chart	`helm create mychart`
render without installing	`helm template name ./chart`
lint	`helm lint ./chart`
install or upgrade	`helm upgrade --install name ./chart -f values.yaml`
list releases	`helm list`
revision history	`helm history name`
roll back	`helm rollback name <revision>`
show values	`helm show values repo/chart`
remove	`helm uninstall name`

13.9 Deploy and roll back

Task	Command
rollout state	`kubectl rollout status deploy/web`
revisions	`kubectl rollout history deploy/web`
pause	`kubectl rollout pause deploy/web`
resume	`kubectl rollout resume deploy/web`
undo	`kubectl rollout undo deploy/web`
new image	`kubectl set image deploy/web web=nginx:1.28`
scale	`kubectl scale deploy/web --replicas=5`

13.10 Troubleshooting order

1. `kubectl get pods -o wide` for status and restarts.
2. `kubectl describe pod <pod>` for events and mount errors.
3. `kubectl logs <pod> -c <container> [--previous]` for the application.
4. `kubectl get events --sort-by=.lastTimestamp` for the cluster diary.
5. `kubectl top pods` and `kubectl get pvc` for resources and storage.

Final test

1. Which component is the only one that reads and writes the cluster state? A) kube-scheduler B) kube-apiserver C) kubelet D) kube-proxy

2. Which block of an object is filled in by the control plane? A) `spec` B) `status` C) `metadata` D) `kind`
3. What is the smallest object you can deploy? A) a container B) a pod C) a ReplicaSet D) a node
4. Which two things do all containers in one pod share? A) the image and the user B) the network namespace and mounted volumes C) the node and the namespace D) the probes and the limits
5. True or false: a bare pod is recreated after it is deleted.
6. Which object owns the pods in a Deployment? A) the Deployment, directly B) a ReplicaSet C) a Service D) the scheduler
7. What matches a ReplicaSet to its pods? A) names B) label selectors C) owner references only D) the namespace
8. Which Service type is reachable only inside the cluster? A) `NodePort` B) `ClusterIP` C) `LoadBalancer` D) `ExternalName`
9. What does a headless Service do? A) blocks all traffic B) returns pod addresses through DNS C) creates a public IP D) removes the endpoints
10. Which object holds non-confidential configuration? A) Secret B) ConfigMap C) PersistentVolume D) LimitRange
11. Which Secret field accepts plain-text values before encoding? A) `data` B) `stringData` C) `binaryData` D) `values`
12. Which field of a pod references persistent storage? A) `volumes[].persistentVolumeClaim.claimName` B) `volumes[].hostPath.path` C) `volumes[].emptyDir` D) `spec.storageClassName`
13. Which access mode allows many nodes to write at the same time? A) `ReadWriteOnce` B) `ReadOnlyMany` C) `ReadWriteMany` D) `ReadWriteOncePod`
14. Which StatefulSet field creates one claim per replica? A) `volumeMounts` B) `volumeClaimTemplates` C) `serviceName` D) `claims`
15. Which probe removes a pod from Service endpoints when it fails? A) liveness B) readiness C) startup D) all three
16. What happens when a container exceeds its memory limit? A) it is throttled B) it is killed with an OOM error C) nothing D) the node is tainted
17. CPU throttling is the effect of exceeding which field? A) `requests.cpu` B) `limits.cpu` C) `requests.memory` D) `limits.memory`
18. A pod has requests but no limits on every container. Which QoS class is it? A) Guaranteed B) Burstable C) BestEffort D) Managed
19. Which label makes a namespace reject pods that run as root? A) `pod-security.kubernetes.io/enforce: restricted` B) `pod-security.kubernetes.io/audit: restricted` C) `pod-security.kubernetes.io/warn: restricted` D) `security.kubernetes.io/enforce: baseline`
20. What must exist before an Ingress object routes traffic? A) a Service of type LoadBalancer B) an ingress controller C) a PersistentVolume D) a service account

21. Which field selects the controller for an Ingress? A) `spec.className` B) `spec.ingressClassName` C) `metadata.controller` D) `spec.type`
22. Which file names and versions a Helm chart? A) `values.yaml` B) `Chart.yaml` C) `templates.yaml` D) `Chartfile`
23. What does `kubectl rollout undo deployment/web` produce? A) a deletion of the deployment B) a new revision built from an older template C) a restart with the same image D) a scale to zero
24. Which two commands promote a canary that shares one Service selector? A) scale the canary up and the stable down B) scale the stable up and the canary down C) delete the Service and re-create it D) patch the Service clusterIP
25. A pod is stuck in `CrashLoopBackOff`. Which command shows the logs of the run that crashed? A) `kubectl logs pod` B) `kubectl logs pod --previous` C) `kubectl describe service` D) `kubectl top pod`

14.1 Final test answers

1. B - the API server is the only component with access to etcd, which keeps validation and auditing in one place.
2. B - `status` is the report the system writes; you only read it.
3. B - a pod is the smallest schedulable unit and wraps the containers.
4. B - containers of one pod share the network namespace and any volumes the pod declares.
5. False - a bare pod disappears for good; controllers are what bring work back.
6. B - the Deployment controller manages a ReplicaSet, and the ReplicaSet creates the pods.
7. B - selectors are the loose binding between controller and pod.
8. B - ClusterIP is the internal-only default.
9. B - `clusterIP: None` makes DNS return the pod addresses.
10. B - ConfigMaps hold non-confidential configuration.
11. B - `stringData` is encoded by the server before storage.
12. A - pods reference a claim, which is bound to a volume.
13. C - `ReadWriteMany` is the multi-writer mode, usually file-based.
14. B - claim templates are stamped per replica with the stateful name.
15. B - readiness gates the endpoints; liveness restarts containers.
16. B - memory cannot be throttled, so the runtime kills the process.
17. B - the CPU limit is enforced by throttling, not by killing.
18. B - some requests without matching limits is Burstable.
19. A - only the `enforce` label rejects; `audit` records and `warn` informs.
20. B - an Ingress is configuration for a controller, so the controller must be installed and running.
21. B - `ingressClassName` names the controller that must implement the rules.
22. B - `Chart.yaml` carries `apiVersion: v2`, `version` and `appVersion`.

23. B - the rollback is a new revision whose template comes from the past.
24. A - traffic follows the replica counts while the Service keeps the same selector.
25. B - `--previous` reads the logs of the container that crashed before the restart.

Sources

Consulted while writing this volume:

- Kubernetes tutorials, the syllabus followed here: https://kubernetes.io/docs/tutorials/
- Hello Minikube: https://kubernetes.io/docs/tutorials/hello-minikube/
- Kubernetes basics, create a cluster: https://kubernetes.io/docs/tutorials/kubernetes-basics/create-cluster/cluster-intro/
- Kubernetes basics, deploy an app: https://kubernetes.io/docs/tutorials/kubernetes-basics/deploy-app/deploy-intro/
- Kubernetes basics, explore your app: https://kubernetes.io/docs/tutorials/kubernetes-basics/explore/explore-intro/
- Kubernetes basics, expose your app: https://kubernetes.io/docs/tutorials/kubernetes-basics/expose/expose-intro/
- Kubernetes basics, scale your app: https://kubernetes.io/docs/tutorials/kubernetes-basics/scale/scale-intro/
- Kubernetes basics, update your app: https://kubernetes.io/docs/tutorials/kubernetes-basics/update/update-intro/
- Cluster architecture: https://kubernetes.io/docs/concepts/architecture/architecture/
- Nodes: https://kubernetes.io/docs/concepts/architecture/nodes/
- Pods: https://kubernetes.io/docs/concepts/workloads/pods/
- Deployments: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/
- ReplicaSets: https://kubernetes.io/docs/concepts/workloads/controllers/replicaset/
- StatefulSets: https://kubernetes.io/docs/concepts/workloads/controllers/statefulset/
- Services: https://kubernetes.io/docs/concepts/services-networking/service/
- DNS for services and pods: https://kubernetes.io/docs/concepts/services-networking/dns-pod-service/
- Ingress: https://kubernetes.io/docs/concepts/services-networking/ingress/
- ConfigMaps: https://kubernetes.io/docs/concepts/configuration/configmap/
- Secrets: https://kubernetes.io/docs/concepts/configuration/secret/
- Persistent volumes: https://kubernetes.io/docs/concepts/storage/persistent-volumes/
- Configure liveness, readiness and startup probes: https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/
- Resource management for pods and containers: https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/

- Namespaces: https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/
- Using RBAC authorisation: https://kubernetes.io/docs/reference/access-authn-authz/rbac/
- Pod Security Standards: https://kubernetes.io/docs/concepts/security/pod-security-standards/
- Helm documentation: https://helm.sh/docs/
- `kubectl` reference: https://kubernetes.io/docs/reference/kubectl/
- Debug running pods: https://kubernetes.io/docs/tasks/debug/debug-application/debug-running-pod/
- Logging architecture: https://kubernetes.io/docs/concepts/cluster-administration/logging/
- The Kubernetes dashboard and add-ons, including metrics-server: https://kubernetes.io/docs/tasks/access-application-cluster/web-ui-dashboard/